```
// ===== 文件：lib/main.dart =====
0001  import 'dart:ui';
0002  import 'package:flutter/material.dart';
0003  import 'package:flutter/services.dart';
0004  import 'core/database/storage_service.dart';
0005  import 'core/theme/cyber_colors.dart';
0006  import 'core/utils/cyber_logger.dart';
0007  import 'features/settings/presentation/widgets/privacy_agreement_modal.dart';
0008  import 'features/audio/services/ambient_service.dart';
0009  import 'features/audio/services/sfx_service.dart';
0010  import 'features/dashboard/presentation/dashboard_screen.dart';
0011  import 'features/library/domain/book_model.dart';
0012  import 'features/reader/providers/reader_provider.dart';
0013  import 'features/settings/providers/settings_provider.dart';
0014  import 'package:flutter_riverpod/flutter_riverpod.dart' as riverpod;
0015  import 'shared/widgets/tts_error_listener.dart';
0016  import 'features/audio/services/tts_engine_service.dart';
0018  final GlobalKey<NavigatorState> globalNavigatorKey =
0019      GlobalKey<NavigatorState>();
0021  Future<void> main() async {
0022    WidgetsFlutterBinding.ensureInitialized();
0024    // 全局错误捕获锚点（先注册，确保 Sentry 初始化前的错误也能记录）
0025    FlutterError.onError = CyberLogger.recordFlutterError;
0026    PlatformDispatcher.instance.onError = CyberLogger.recordPlatformError;
0028    // 启动前初始化持久化层与音效引擎
0029    await StorageService.init();
0030    try {
0031      await SfxService.init();
0032    } catch (e, st) {
0033      CyberLogger.recordFlutterError(
0034        FlutterErrorDetails(exception: e, stack: st, library: 'SfxService'),
0035      );
0036    }
0038    // 初始化环境背景音服务
0039    try {
0040      await AmbientService.init();
0041    } catch (e, st) {
0042      CyberLogger.recordFlutterError(
0043        FlutterErrorDetails(exception: e, stack: st, library: 'AmbientService'),
0044      );
0045    }
0047    // 通过 CyberLogger 初始化 Sentry 并启动 App
0048    // DSN 通过 --dart-define=SENTRY_DSN=https://... 注入，空时静默跳过
0049    await CyberLogger.initSentry(
0050      () async => runApp(
0051        const riverpod.ProviderScope(
0052          child: YueYouApp(),
0053        ),
0054      ),
0055    );
```

```
0056 }
0058 class YueYouApp extends riverpod.ConsumerWidget {
0059   const YueYouApp({super.key});
0061   @override
0062   Widget build(BuildContext context, riverpod.WidgetRef ref) {
0063     return const _Bootstrapper();
0064   }
0065 }
0067 class _Bootstrapper extends riverpod.ConsumerStatefulWidget {
0068   const _Bootstrapper();
0070   @override
0071   riverpod.ConsumerState<_Bootstrapper> createState() => _BootstrapperState();
0072 }
0074 class _BootstrapperState extends riverpod.ConsumerState<_Bootstrapper>
0075     with WidgetsBindingObserver {
0076   bool _booted = false;
0078   /// 记录上一次 SettingsProvider 的 ambient 状态，用于增量对比
0079   bool? _lastAmbientEnabled;
0080   double? _lastAmbientVol;
0081   String? _lastAmbientStyle;
0083   @override
0084   void initState() {
0085     super.initState();
0086     WidgetsBinding.instance
0087       ..addObserver(this)
0088       ..addPostFrameCallback((_) => _checkPrivacyAndBootstrap());
0089   }
0091   @override
0092   void didChangeDependencies() {
0093     super.didChangeDependencies();
0094     // 每当 SettingsProvider 变化时同步 AmbientService
0095     final settings = ref.watch(settingsProvider);
0096     _syncAmbient(settings);
0097   }
0099   /// 同步 SettingsProvider 的 ambient 设置到 AmbientService
0100   void _syncAmbient(SettingsProvider settings) {
0101     if (settings.ambientEnabled != _lastAmbientEnabled) {
0102       _lastAmbientEnabled = settings.ambientEnabled;
0103       AmbientService.setEnabled(settings.ambientEnabled);
0104     }
0105     if (settings.ambientVol != _lastAmbientVol) {
0106       _lastAmbientVol = settings.ambientVol;
0107       AmbientService.setVolume(settings.ambientVol);
0108     }
0109     if (settings.ambientStyle != _lastAmbientStyle) {
0110       _lastAmbientStyle = settings.ambientStyle;
0111       AmbientService.setStyle(settings.ambientStyle);
0112     }
0113   }
0115   /// App 生命周期：切后台暂停，切前台恢复
```

```
  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    switch (state) {
      case AppLifecycleState.paused:
      case AppLifecycleState.inactive:
        AmbientService.pause();
      case AppLifecycleState.resumed:
        AmbientService.resume();
      case AppLifecycleState.detached:
      case AppLifecycleState.hidden:
        AmbientService.dispose();
    }
  }
  Future<void> _checkPrivacyAndBootstrap() async {
    if (!mounted) return;
    if (!StorageService.hasAgreedPrivacy()) {
      // 等待 MaterialApp 内部 Navigator 挂载完成（最多 5 次，间隔 50ms）
      // 避免首帧时 globalNavigatorKey 尚未绑定导致弹窗被静默跳过
      BuildContext? navContext;
      for (int i = 0; i < 5; i++) {
        navContext = globalNavigatorKey.currentContext;
        if (navContext != null) break;
        await Future<void>.delayed(const Duration(milliseconds: 50));
        if (!mounted) return;
      }
      if (navContext == null) {
        // 极端情况：Navigator 始终未挂载，记录日志后退出，绝不进入未授权状态
        CyberLogger.captureWarning(
          Exception('隐私弹窗 navContext 重试 5 次仍为 null'),
          tag: 'privacy',
          extra: {'context': '_checkPrivacyAndBootstrap'},
        );
        SystemNavigator.pop();
        return;
      }
      // navContext 来自 globalNavigatorKey，由 Navigator 自身管理生命周期，
      // 不依赖 _BootstrapperState.mounted；上方循环已做 mounted 守卫。
      // ignore: use_build_context_synchronously
      final agreed = await showPrivacyAgreementModal(navContext);
      if (!mounted) return;
      if (agreed) {
        await StorageService.setHasAgreedPrivacy(true);
      } else {
        return;
      }
    }
    await _bootstrap();
  }
  Future<void> _bootstrap() async {
    if (_booted) return;
```

```
    _booted = true;
    final reader = ref.read(readerProvider);
    final currentNovelId = StorageService.getCurrentNovelId();
    if (currentNovelId == null) return;
    final content = await StorageService.loadBookContent(currentNovelId);
    if (!mounted || content == null) return;
    final List<dynamic> rawLines = content['lines'] as List<dynamic>? ?? [];
    final List<dynamic> rawChapters =
        content['chapters'] as List<dynamic>? ?? [];
    final chapters = rawChapters
        .map((e) => ChapterModel.fromJson(e as Map<String, dynamic>))
        .toList();
    final lines = rawLines.map((e) => e.toString()).toList();
    final int initialIndex = StorageService.getCurrentNovelIndex();
    await reader.loadPreparedBook(
      lines,
      bookId: currentNovelId,
      chapters: chapters,
      initialIndex: initialIndex,
    );
  }
  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      navigatorKey: globalNavigatorKey,
      title: '阅游 YueYou',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        scaffoldBackgroundColor: CyberColors.background,
        colorScheme: const ColorScheme.dark(
          primary: CyberColors.neonGreen,
          secondary: CyberColors.neonPink,
        ),
        useMaterial3: true,
      ),
      builder: (context, child) => Listener(
        onPointerDown: (_) => ref.read(ttsEngineProvider).notifyUserActivity(),
        behavior: HitTestBehavior.translucent,
        child: TtsErrorListener(child: child!),
      ),
      home: const DashboardScreen(),
    );
  }
}
```

```
// ===== 文件：lib/core/config/tts_config.dart =====
0001  /// TTS 配置类
0002  ///
0003  /// 所有服务器地址通过 `--dart-define` 编译时注入，严禁硬编码 IP 或域名。
0004  class TtsConfig {
0005    final String serverUrl;
0006    final Duration requestTimeout;
0007    final int maxRetries;
0008    final Duration baseRetryDelay;
0009    final int maxPrefetchQueue;
0011    const TtsConfig({
0012      required this.serverUrl,
0013      this.requestTimeout = const Duration(seconds: 8),
0014      this.maxRetries = 2,
0015      this.baseRetryDelay = const Duration(milliseconds: 800),
0016      this.maxPrefetchQueue = 6,
0017    });
0019    /// 编译时注入的 TTS 服务器地址（--dart-define=TTS_SERVER_URL=https://...）
0020    static const String _ttsServerUrl = String.fromEnvironment(
0021      'TTS_SERVER_URL',
0022      defaultValue: 'https://hclstudio.cn/api/v1/tts',
0023    );
0025    /// 编译时注入的书籍 API 基础地址（--dart-define=BOOK_API_BASE=https://...）
0026    static const String bookApiBase = String.fromEnvironment(
0027      'BOOK_API_BASE',
0028      defaultValue: 'https://hclstudio.cn/api/v1',
0029    );
0031    // ── 网络层超时常量 ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────
0033    /// TTS 本地引擎朗读最长等待时间（防止 FlutterTts 卡死）
0034    static const Duration ttsLocalSpeakTimeout = Duration(seconds: 60);
0036    /// TTS HTTP 下载连接与传输超时
0037    static const Duration ttsDownloadTimeout = Duration(seconds: 15);
0039    /// TTS POST 请求连接超时
0040    static const Duration ttsPostConnectionTimeout = Duration(seconds: 10);
0042    /// TTS POST 响应超时
0043    static const Duration ttsPostResponseTimeout = Duration(seconds: 15);
0045    /// 书籍目录 / 章节 API 请求超时
0046    static const Duration bookApiTimeout = Duration(seconds: 4);
0048    /// 书籍章节 CDN 下载超时
0049    static const Duration bookCdnDownloadTimeout = Duration(seconds: 15);
0051    /// 当前环境配置（编译时确定，零运行时开销）
0052    static const TtsConfig current = TtsConfig(
0053      serverUrl: _ttsServerUrl,
0054    );
0055  }

// ===== 文件：lib/core/constants/book_constants.dart =====
0001  /// 书籍相关常量定义
0002  class BookConstants {
0003    /// 内置默认书籍 ID（西游记），int 类型对齐 BookModel.id
```

```
0004    static const int defaultBookId = 999;
0006    /// 内置默认书籍标识字符串（用于分章缓存目录命名）
0007    static const String defaultBookKey = 'xiyouji';
0009    /// 内置默认书籍标题
0010    static const String defaultBookTitle = '西游记';
0012    /// 内置默认书籍作者
0013    static const String defaultBookAuthor = '吴承恩';
0015    /// 内置默认书籍总章节数
0016    static const int defaultTotalChapters = 100;
0018    /// 《西游记》100 回章节标题（离线降级用，索引 0-based 对应第1回至第100回）
0019    static const List<String> xiyoujiChapterTitles = [
0020      '第一回 灵根育孕源流出 心性修持大道生',
0021      '第二回 悟彻菩提真妙理 断魔归本合元神',
0022      '第三回 四海千山皆拱伏 九幽十类尽除名',
0023      '第四回 官封弼马心何足 名注齐天意未宁',
0024      '第五回 乱蟠桃大圣偷丹 反天宫诸神捉怪',
0025      '第六回 观音赴会问原因 小圣施威降大圣',
0026      '第七回 八卦炉中逃大圣 五行山下定心猿',
0027      '第八回 我佛造经传极乐 观音奉旨上长安',
0028      '第九回 袁守诚妙算无私曲 老龙王拙计犯天条',
0029      '第十回 二将军宫门镇鬼 唐太宗地府还魂',
0030      '第十一回 还受生唐王遵善果 度孤魂萧瑀正空门',
0031      '第十二回 玄奘秉诚建大会 观音显象化金蝉',
0032      '第十三回 陷虎穴金星解厄 双叉岭伯钦留僧',
0033      '第十四回 心猿归正 六贼无踪',
0034      '第十五回 蛇盘山诸神暗佑 鹰愁涧意马收缰',
0035      '第十六回 观音院僧谋宝贝 黑风山怪窃袈裟',
0036      '第十七回 孙行者大闹黑风山 观世音收伏熊罴怪',
0037      '第十八回 观音院唐僧脱难 高老庄大圣除魔',
0038      '第十九回 云栈洞悟空收八戒 浮屠山玄奘受心经',
0039      '第二十回 黄风岭唐僧有难 半山中八戒争先',
0040      '第二十一回 护法设庄留大圣 须弥灵吉定风魔',
0041      '第二十二回 八戒大战流沙河 木叉奉法收悟净',
0042      '第二十三回 三藏不忘本 四圣试禅心',
0043      '第二十四回 万寿山大仙留故友 五庄观行者窃人参',
0044      '第二十五回 镇元仙赶捉取经僧 孙行者大闹五庄观',
0045      '第二十六回 孙悟空三岛求方 观世音甘泉活树',
0046      '第二十七回 尸魔三戏唐三藏 圣僧恨逐美猴王',
0047      '第二十八回 花果山群妖聚义 黑松林三藏逢魔',
0048      '第二十九回 脱难江流来国土 承恩八戒转山林',
0049      '第三十回 邪魔侵正法 意马忆心猿',
0050      '第三十一回 猪八戒义激猴王 孙行者智降妖怪',
0051      '第三十二回 平顶山功曹传信 莲花洞木母逢灾',
0052      '第三十三回 外道迷真性 元神助本心',
0053      '第三十四回 魔王巧算困心猿 大圣腾挪骗宝贝',
0054      '第三十五回 外道施威欺正性 心猿获宝伏邪魔',
0055      '第三十六回 心猿正处诸缘伏 劈破旁门见月明',
0056      '第三十七回 鬼王夜谒唐三藏 悟空神化引婴儿',
0057      '第三十八回 婴儿问母知邪正 金木参玄见假真',
0058      '第三十九回 一粒金丹天上得 三年故主世间生',
```

```
0059        '第四十回 婴儿戏化禅心乱 猿马刀圭木母空',
0060        '第四十一回 心猿遭火败 木母被魔擒',
0061        '第四十二回 大圣殷勤拜南海 观音慈善缚红孩',
0062        '第四十三回 黑河妖孽擒僧去 西洋龙子捉鼍回',
0063        '第四十四回 法身元运逢车力 心正妖邪度脊关',
0064        '第四十五回 三清观大圣留名 车迟国猴王显法',
0065        '第四十六回 外道弄强欺正法 心猿显圣灭诸邪',
0066        '第四十七回 圣僧夜阻通天水 金木垂慈救小童',
0067        '第四十八回 魔弄寒风飘大雪 僧思拜佛履层冰',
0068        '第四十九回 三藏有灾沉水宅 观音救难现鱼篮',
0069        '第五十回 情因旧恨生灾毒 心主遭魔幸破光',
0070        '第五十一回 心猿空用千般计 水火无功难炼魔',
0071        '第五十二回 悟空大闹金兜洞 如来暗示主人公',
0072        '第五十三回 禅主吞餐怀鬼孕 黄婆运水解邪胎',
0073        '第五十四回 法性西来逢女国 心猿定计脱烟花',
0074        '第五十五回 色邪淫戏唐三藏 性正归真胜女色',
0075        '第五十六回 神狂诛草寇 道昧放心猿',
0076        '第五十七回 真行者落伽山诉苦 假猴王水帘洞誊文',
0077        '第五十八回 二心搅乱大乾坤 一体难修真寂灭',
0078        '第五十九回 唐三藏路阻火焰山 孙行者一调芭蕉扇',
0079        '第六十回 牛魔王罢战赴华筵 孙行者二调芭蕉扇',
0080        '第六十一回 猪八戒助力败魔王 孙行者三调芭蕉扇',
0081        '第六十二回 涤垢洗心惟扫塔 缚魔归正乃修身',
0082        '第六十三回 二僧荡涤波月洞 胜法降伏无底船',
0083        '第六十四回 荆棘岭悟能努力 木仙庵三藏谈诗',
0084        '第六十五回 妖邪假设小雷音 四众皆遭大厄难',
0085        '第六十六回 诸神遭毒手 弥勒缚妖魔',
0086        '第六十七回 拯救驼罗庄 消除疥癞病',
0087        '第六十八回 朱紫国唐僧论前世 孙行者施为三折肱',
0088        '第六十九回 心主夜间修药物 君王筵上论妖邪',
0089        '第七十回 妖魔宝放烟沙火 悟空计盗紫金铃',
0090        '第七十一回 行者假名降怪狐 观音现像伏金毛',
0091        '第七十二回 盘丝洞七情迷本 濯垢泉八戒忘形',
0092        '第七十三回 情因旧恨盘丝洞 壁脱香风蜈蚣精',
0093        '第七十四回 长庚传报魔头狠 行者施为变化能',
0094        '第七十五回 心猿钻入魔王腹 将法三番数字成',
0095        '第七十六回 心神居舍魔归性 木母同降怪体真',
0096        '第七十七回 群魔欺本性 一体拜真如',
0097        '第七十八回 比丘怜子遣阴神 金殿识魔谈道德',
0098        '第七十九回 寻洞擒妖逢老寿 当朝正主救婴儿',
0099        '第八十回 姹女育阳求配偶 心猿护主识妖邪',
0100        '第八十一回 镇海寺心猿知怪 黑松林三众寻师',
0101        '第八十二回 姹女求阳 元神护道',
0102        '第八十三回 心猿识得丹头 姹女还归本性',
0103        '第八十四回 难灭伽持圆大觉 法王成正体天然',
0104        '第八十五回 心猿妒木母 魔主计吞禅',
0105        '第八十六回 木母助威征怪物 金公施法灭妖邪',
0106        '第八十七回 凤仙郡冒天止雨 孙大圣劝善施霖',
0107        '第八十八回 禅到玉华施法会 心猿木母授门人',
0108        '第八十九回 黄狮精虎力大 白鹿精太乙真',
```

```
      '第九十回 师狮授受同归一 盗宝欺人反受殃',
      '第九十一回 金平府元夜观灯 玄英洞唐僧供状',
      '第九十二回 三僧大战青龙山 四星挟捉犀牛怪',
      '第九十三回 给孤园问古怀乡 天竺国朝王遇偶',
      '第九十四回 四僧宴乐御花园 一怪空怀情欲喜',
      '第九十五回 假合真形擒玉兔 真阴归正会灵元',
      '第九十六回 寇员外喜待高僧 唐长老不贪富贵',
      '第九十七回 金酬外护遭魔蛰 圣显幽魂救本原',
      '第九十八回 猿熟马驯方论正 功完行满见真如',
      '第九十九回 九九数完魔灭尽 三三行满道归根',
      '第一百回 径回东土 五圣成真',
    ];
  }

// ===== 文件：lib/core/constants/cyber_error_messages.dart =====
  class CyberErrorMessages {
    // ==========================================
    //  网络与云端通讯错误（屏蔽底层实现细节）
    // ==========================================
    static const String ttsNodeUnresponsive = '语音服务暂不可用，请检查网络后重试';
    static const String ttsConnectTimeout = '服务连接超时，请检查网络设置';
    static const String ttsRequestTimeout = '请求处理超时，请稍后重试';
    static const String ttsAudioLoadTimeout = '语音加载缓慢，为您切换至本地语音';
    static String ttsServerErrorCode(int code) => '服务器繁忙，请稍后重试（错误码：$code)';
    // ==========================================
    //  降级与兜底提醒（用户友好的状态变更）
    // ==========================================
    static const String ttsFallbackDisconnected = '云端连接因网络波动断开，已自动切换为本地语音';
    static const String ttsFallbackTimeout = '云端资源加载失败，正在使用本地语音播放';
    // ==========================================
    //  协议与解析异常（隐藏服务架构与堆栈）
    // ==========================================
    static const String ttsInvalidFormat = '语音数据异常，请确保应用为最新版或稍后重试';
    static const String ttsNotJsonObject = '语音服务暂不支持该解析，请稍后重试';
    static String ttsMissingUrl(String body) => '获取语音地址失败，请重新尝试';
    static String ttsMissingUrlTest(String body) => '语音服务通道发生异常，请联系开发者获取支持';
    // ==========================================
    //  业务逻辑与系统状态提示
    // ==========================================
    static const String ttsRequireBookFirst = '请先导入一部书籍，再开启语音朗读';
    static const String ttsNoContentRequiresBook = '当前没有内容可以朗读，请先选择一部书籍';
    static const String ttsIdleTimeout = '您已经休息很久啦，为了省电，语音已自动暂停';
    static const String ttsAudioParamFailed = '语音参数调节失败，请重启功能重试';
    // ==========================================
    //  本地数据与文件导入
    // ==========================================
    static const String importFormatFailed = '不支持该书籍的格式，请确认是否为文本文件';
    static const String importReadPathFailed = '无法读取文件路径，请检查您的存储权限';
    static const String testFailedUnresponsive = '连通性测试未通过：网络连接异常，服务不可用';
    static String importFileTooLarge(int maxMb) =>
```

```
      '为了保证流畅阅读，暂不支持体积超过 ${maxMb}MB 的书籍，建议分卷导入';
}
```

// ===== 文件：lib/core/database/i_storage_service.dart =====

```
/// 本地持久化服务的统一接口定义（依赖倒置原则）
///
/// 所有业务逻辑层应依赖本接口，而非直接引用 [StorageService] 静态类。
/// 这样可在测试中注入 Mock 实现，在生产环境中注入 [LocalStorageService]，
/// 实现业务逻辑与具体存储实现的完全解耦。
///
/// ## 设计约束
/// - 本接口位于 `core/database/`，仅允许被 `features/*/providers/` 消费。
/// - 严禁在 `presentation/` 层直接调用任何存储接口。
/// - 所有写操作均为 async（`Future<void>`），读操作可同步或异步。
abstract interface class IStorageService {
  // ── 初始化 ──────────────────────────────────────────────────────────
  /// 初始化底层存储引擎（应在 [runApp] 前调用一次）
  Future<void> init();
  // ── 游戏状态 ────────────────────────────────────────────────────────
  /// 保存完整的 2048 游戏快照
  ///
  /// - [board]：4×4 棋盘（`null` 表示空格，非 `null` 为 `{id, value}`）
  /// - [score]：当前全盘总分（全盘求和法，非累加法）
  /// - [combo]：当前连击数
  /// - [bestScore]：历史最佳得分
  /// - [maxCombo]：历史最大连击
  /// - [novelIndex]：当前关联的小说索引
  /// - [currentNovelId]：当前关联的小说 ID（可为 null）
  Future<void> saveGameState({
    required List<List<Map<String, dynamic>?>> board,
    required int score,
    required int combo,
    required int bestScore,
    required int maxCombo,
    required int novelIndex,
    String? currentNovelId,
  });
  /// 读取游戏快照，返回 `null` 表示无存档或存档损坏
  Map<String, dynamic>? loadGameState();
  /// 读取历史最佳得分（无存档时返回 0）
  int loadBestScore();
  /// 读取历史最大连击数（无存档时返回 0）
  int loadMaxCombo();
  // ── 书架元数据 ──────────────────────────────────────────────────────
  /// 保存书架元数据列表（全量覆盖写入）
  ///
  /// 每个元素为书籍元数据 Map，至少包含 `id`、`title`、`total` 字段。
  Future<void> saveBookshelf(List<Map<String, dynamic>> shelf);
  /// 读取书架元数据列表（损坏或空时返回空列表）
  List<Map<String, dynamic>> loadBookshelf();
```

```
0057    // ─── 阅读进度 ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────
0059    /// 更新指定书籍的阅读进度
0060    ///
0061    /// - [bookId]：书籍唯一标识
0062    /// - [cursor]：当前行号（0-based）
0063    /// - [total]：总行数（≤ 0 时忽略，防止除零）
0064    Future<void> updateReadingRecord(String bookId, int cursor, int total);
0066    /// 读取指定书籍的阅读进度
0067    ///
0068    /// 返回 `{cursor, total, percent}`，无记录时返回默认零进度。
0069    Map<String, dynamic> getReadingRecord(String bookId);
0071    /// 删除指定书籍的阅读进度记录
0072    Future<void> deleteReadingRecord(String bookId);
0074    // ─── 当前小说状态 ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
0076    /// 设置当前激活的小说 ID（传入 `null` 时清除）
0077    Future<void> setCurrentNovelId(String? id);
0079    /// 读取当前激活的小说 ID（未设置时返回 `null`）
0080    String? getCurrentNovelId();
0082    /// 读取当前激活的小说在书架中的索引（未设置时返回 0）
0083    int getCurrentNovelIndex();
0085    /// 设置当前激活的小说在书架中的索引
0086    Future<void> setCurrentNovelIndex(int index);
0088    // ─── 书籍正文内容 ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
0090    /// 将书籍正文按行存储为 JSON 文件（大文件，存入 Documents 目录）
0091    ///
0092    /// - [bookId]：书籍唯一标识（作为文件名）
0093    /// - [lines]：全文按行分割后的字符串列表
0094    /// - [chapters]：章节目录，每项至少含 `title`、`lineIndex` 字段
0095    Future<void> saveBookContent(
0096      String bookId, {
0097      required List<String> lines,
0098      required List<Map<String, dynamic>> chapters,
0099    });
0101    /// 读取书籍正文内容
0102    ///
0103    /// 返回 `{lines: List<String>, chapters: List<Map>}`，
0104    /// 文件不存在或 JSON 损坏时返回 `null`。
0105    Future<Map<String, dynamic>?> loadBookContent(String bookId);
0107    /// 删除指定书籍的正文文件（文件不存在时静默忽略）
0108    Future<void> deleteBookContent(String bookId);
0110    // ─── 全局设置 ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────
0112    /// 读取音效开关（默认 `true`）
0113    bool getSettingSound();
0115    /// 设置音效开关并持久化
0116    Future<void> setSettingSound(bool v);
0118    /// 读取 TTS 朗读开关（默认 `true`）
0119    bool getSettingStoryTts();
0121    /// 设置 TTS 朗读开关并持久化
0122    Future<void> setSettingStoryTts(bool v);
0124    /// 读取 TTS 音色 ID（默认 `'zh-CN-XiaoxiaoNeural'`）
```

```
  String getSettingVoice();

  /// 设置 TTS 音色 ID 并持久化
  Future<void> setSettingVoice(String v);

  /// 读取空闲超时（分钟，默认 1）
  int getSettingIdleTimeout();

  /// 设置空闲超时并持久化
  Future<void> setSettingIdleTimeout(int v);

  /// 读取 TTS 播放倍速（默认 `1.0`）
  double getSettingTtsRate();

  /// 设置 TTS 播放倍速并持久化
  Future<void> setSettingTtsRate(double v);

  /// 读取环境音量（默认 `0.5`，范围 `0.0`–`1.0`）
  double getSettingAmbientVol();

  /// 设置环境音量并持久化
  Future<void> setSettingAmbientVol(double v);

  /// 读取环境背景音乐开关（默认 `true`）
  bool getSettingAmbientEnabled();

  /// 设置环境背景音乐开关并持久化
  Future<void> setSettingAmbientEnabled(bool v);

  // ─── 隐私协议 ──────────────────────────────────────────────────────────────────

  /// 读取用户是否已同意隐私协议（默认 `false`）
  bool hasAgreedPrivacy();

  /// 设置隐私协议同意状态并持久化
  Future<void> setHasAgreedPrivacy(bool v);

  // ─── 粘性位：用户是否曾主动选择过书籍 ──────────────────────────────────────────

  /// 读取用户是否曾主动选择过书籍（默认 `false`）
  ///
  /// 用于区分"从未有过书籍的新用户"与"删掉所有书籍的老用户"，
  /// 仅前者在书架为空时触发内置默认书籍自动注入。
  bool hasSelectedBook();

  /// 设置粘性位并持久化
  Future<void> setHasSelectedBook(bool v);

  // ─── 分章文本缓存（文件路径，非 SharedPreferences）──────────────────────────

  /// 将指定书籍的某章节纯文本写入本地文件
  ///
  /// 文件路径：`{documentsDir}/books/chapters/{bookId}/{chapterIndex:03d}.txt`
  Future<void> saveChapterCache(String bookId, int chapterIndex, String text);

  /// 读取指定书籍章节的本地缓存文本
  ///
  /// 文件不存在时返回 `null`。
  Future<String?> loadChapterCache(String bookId, int chapterIndex);

  /// 删除指定书籍所有已缓存章节文件
  Future<void> clearChapterCache(String bookId);

  /// LRU 清理：仅保留 `currentChapterIndex ± keepAround` 范围内的章节缓存
  ///
  /// 用于防止长时间阅读后缓存文件堆积。
  Future<void> pruneChapterCache(
    String bookId,
    int currentChapterIndex, {
    int keepAround = 3,
```

```
  });
  // ─── 书目录缓存（文件路径）──────────────────────────────────────────────────
  /// 将书目录（章节标题 + 索引列表）序列化写入本地文件
  ///
  /// 文件路径：`{documentsDir}/books/catalogs/{bookId}_catalog.json`
  Future<void> saveBookCatalog(
    String bookId,
    List<Map<String, dynamic>> chapters,
  );
  /// 读取书目录缓存
  ///
  /// 文件不存在或 JSON 损坏时返回 `null`。
  Future<List<Map<String, dynamic>>?> loadBookCatalog(String bookId);
}

// ===== 文件：lib/core/database/local_storage_service.dart =====
import 'package:yueyou/core/database/i_storage_service.dart';
import 'package:yueyou/core/database/storage_service.dart';
/// [IStorageService] 的本地持久化实现（生产环境注入版本）
///
/// 内部将所有调用委托给 [StorageService] 静态方法，保持与现有
/// SharedPreferences + path_provider 存储行为 100% 一致。
///
/// ## 用法
/// ```dart
/// // 在 main.dart 中完成初始化
/// final storage = LocalStorageService();
/// await storage.init();
/// runApp(MultiProvider(providers: [
///   Provider<IStorageService>.value(value: storage),
///   // ... 其他 Provider
/// ]));
/// ```
///
/// ## 测试替换
/// 在测试中注入 [MockStorageService]（实现 [IStorageService]），
/// 无需依赖 SharedPreferences，实现完全隔离。
class LocalStorageService implements IStorageService {
  // ─── 初始化 ──────────────────────────────────────────────────────────────
  @override
  Future<void> init() => StorageService.init();
  // ─── 游戏状态 ──────────────────────────────────────────────────────────────
  @override
  Future<void> saveGameState({
    required List<List<Map<String, dynamic>?>> board,
    required int score,
    required int combo,
    required int bestScore,
    required int maxCombo,
    required int novelIndex,
```

```
0039      String? currentNovelId,
0040    }) =>
0041        StorageService.saveGameState(
0042          board: board,
0043          score: score,
0044          combo: combo,
0045          bestScore: bestScore,
0046          maxCombo: maxCombo,
0047          novelIndex: novelIndex,
0048          currentNovelId: currentNovelId,
0049        );
0051    @override
0052    Map<String, dynamic>? loadGameState() => StorageService.loadGameState();
0054    @override
0055    int loadBestScore() => StorageService.loadBestScore();
0057    @override
0058    int loadMaxCombo() => StorageService.loadMaxCombo();
0060    // ─── 书架元数据 ──────────────────────────────────────────────────────────────────────────────────────────────────────
0062    @override
0063    Future<void> saveBookshelf(List<Map<String, dynamic>> shelf) =>
0064        StorageService.saveBookshelf(shelf);
0066    @override
0067    List<Map<String, dynamic>> loadBookshelf() => StorageService.loadBookshelf();
0069    // ─── 阅读进度 ──────────────────────────────────────────────────────────────────────────────────────────────────────
0071    @override
0072    Future<void> updateReadingRecord(String bookId, int cursor, int total) =>
0073        StorageService.updateReadingRecord(bookId, cursor, total);
0075    @override
0076    Map<String, dynamic> getReadingRecord(String bookId) =>
0077        StorageService.getReadingRecord(bookId);
0079    @override
0080    Future<void> deleteReadingRecord(String bookId) =>
0081        StorageService.deleteReadingRecord(bookId);
0083    // ─── 当前小说状态 ──────────────────────────────────────────────────────────────────────────────────────────────────
0085    @override
0086    Future<void> setCurrentNovelId(String? id) =>
0087        StorageService.setCurrentNovelId(id);
0089    @override
0090    String? getCurrentNovelId() => StorageService.getCurrentNovelId();
0092    @override
0093    int getCurrentNovelIndex() => StorageService.getCurrentNovelIndex();
0095    @override
0096    Future<void> setCurrentNovelIndex(int index) =>
0097        StorageService.setCurrentNovelIndex(index);
0099    // ─── 书籍正文内容 ──────────────────────────────────────────────────────────────────────────────────────────────────
0101    @override
0102    Future<void> saveBookContent(
0103      String bookId, {
0104      required List<String> lines,
0105      required List<Map<String, dynamic>> chapters,
```

```
0106    }) =>
0107        StorageService.saveBookContent(bookId, lines: lines, chapters: chapters);
0109    @override
0110    Future<Map<String, dynamic>?> loadBookContent(String bookId) =>
0111        StorageService.loadBookContent(bookId);
0113    @override
0114    Future<void> deleteBookContent(String bookId) =>
0115        StorageService.deleteBookContent(bookId);
0117    // ——— 全局设置 ———————————————————————————————————————————————————————————————————————————————————————————————————————
0119    @override
0120    bool getSettingSound() => StorageService.getSettingSound();
0122    @override
0123    Future<void> setSettingSound(bool v) => StorageService.setSettingSound(v);
0125    @override
0126    bool getSettingStoryTts() => StorageService.getSettingStoryTts();
0128    @override
0129    Future<void> setSettingStoryTts(bool v) =>
0130        StorageService.setSettingStoryTts(v);
0132    @override
0133    String getSettingVoice() => StorageService.getSettingVoice();
0135    @override
0136    Future<void> setSettingVoice(String v) => StorageService.setSettingVoice(v);
0138    @override
0139    int getSettingIdleTimeout() => StorageService.getSettingIdleTimeout();
0141    @override
0142    Future<void> setSettingIdleTimeout(int v) =>
0143        StorageService.setSettingIdleTimeout(v);
0145    @override
0146    double getSettingTtsRate() => StorageService.getSettingTtsRate();
0148    @override
0149    Future<void> setSettingTtsRate(double v) =>
0150        StorageService.setSettingTtsRate(v);
0152    @override
0153    double getSettingAmbientVol() => StorageService.getSettingAmbientVol();
0155    @override
0156    Future<void> setSettingAmbientVol(double v) =>
0157        StorageService.setSettingAmbientVol(v);
0159    @override
0160    bool getSettingAmbientEnabled() => StorageService.getSettingAmbientEnabled();
0162    @override
0163    Future<void> setSettingAmbientEnabled(bool v) =>
0164        StorageService.setSettingAmbientEnabled(v);
0166    // ——— 隐私协议 ———————————————————————————————————————————————————————————————————————————————————————————————————————
0168    @override
0169    bool hasAgreedPrivacy() => StorageService.hasAgreedPrivacy();
0171    @override
0172    Future<void> setHasAgreedPrivacy(bool v) =>
0173        StorageService.setHasAgreedPrivacy(v);
0175    // ——— 粘性位：用户是否曾主动选择过书籍 ———————————————————————————————————————————————————————————————————————————
0177    @override
```

```
0178    bool hasSelectedBook() => StorageService.hasSelectedBook();
0180    @override
0181    Future<void> setHasSelectedBook(bool v) =>
0182        StorageService.setHasSelectedBook(v);
0184    // ——— 分章文本缓存 ————————————————————————————————————————————————————————————————————————————————————————————————————————————————
0186    @override
0187    Future<void> saveChapterCache(String bookId, int chapterIndex, String text) =>
0188        StorageService.saveChapterCache(bookId, chapterIndex, text);
0190    @override
0191    Future<String?> loadChapterCache(String bookId, int chapterIndex) =>
0192        StorageService.loadChapterCache(bookId, chapterIndex);
0194    @override
0195    Future<void> clearChapterCache(String bookId) =>
0196        StorageService.clearChapterCache(bookId);
0198    @override
0199    Future<void> pruneChapterCache(
0200      String bookId,
0201      int currentChapterIndex, {
0202      int keepAround = 3,
0203    }) =>
0204        StorageService.pruneChapterCache(
0205          bookId,
0206          currentChapterIndex,
0207          keepAround: keepAround,
0208        );
0210    // ——— 书目录缓存 ————————————————————————————————————————————————————————————————————————————————————————————————————————————————
0212    @override
0213    Future<void> saveBookCatalog(
0214      String bookId,
0215      List<Map<String, dynamic>> chapters,
0216    ) =>
0217        StorageService.saveBookCatalog(bookId, chapters);
0219    @override
0220    Future<List<Map<String, dynamic>>?> loadBookCatalog(String bookId) =>
0221        StorageService.loadBookCatalog(bookId);
0222  }

// ===== 文件：lib/core/database/storage_service.dart =====
0001  import 'dart:convert';
0002  import 'dart:io';
0003  import 'package:flutter/foundation.dart';
0004  import 'package:path_provider/path_provider.dart';
0005  import 'package:shared_preferences/shared_preferences.dart';
0006  import 'package:yueyou/core/utils/cyber_logger.dart';
0008  /// 全量本地持久化服务
0009  /// 完整复刻旧版 localStorage + LocalDB.js 的所有存储行为
0010  /// 键名与旧版 JS 100% 对齐，保证未来双端迁移零成本
0011  class StorageService {
0012    // ——— 键名（1:1 对应 JS localStorage key）————————————————————————————————————————————————————————————————————————————————
0013    static const String _kBestScore = 'bestScore_premium';
```

```
  static const String _kMaxCombo = 'maxCombo';
  static const String _kLocalSave = 'local_save_data';
  static const String _kBookshelf = 'local_bookshelf';
  static const String _kReadingRecords = 'reading_records';
  static const String _kCurrentNovelId = 'current_novel_id';
  static const String _kNovelIndex = 'novel_index';
  static const String _kSettingSound = 'setting_sound';
  static const String _kSettingStoryTts = 'setting_story_tts';
  static const String _kSettingVoice = 'setting_voice';
  static const String _kSettingIdleTimeout = 'setting_idle_timeout';
  static const String _kSettingTtsRate = 'setting_tts_rate';
  static const String _kSettingAmbientVol = 'setting_ambient_vol';
  static const String _kSettingAmbientEnabled = 'setting_ambient_enabled';
  static const String _kSettingAmbientStyle = 'setting_ambient_style';
  static const String _kHasAgreedPrivacy = 'has_agreed_privacy';
  static const String _kHasSelectedBook = 'user_has_selected_book';
  static const String _kCurrentChapterIndex = 'current_chapter_index';
  static const String _kSettingAnimationQuality = 'setting_animation_quality';
  static SharedPreferences? _prefs;
  static Future<void> init() async {
    _prefs ??= await SharedPreferences.getInstance();
  }
  /// 测试专用：清除 SharedPreferences 单例缓存，使 [init] 可重新初始化
  /// 生产环境禁止调用
  @visibleForTesting
  static void resetForTesting() => _prefs = null;
  static SharedPreferences get _p {
    assert(_prefs != null, 'StorageService.init() 必须在 runApp 前调用');
    return _prefs!;
  }
  // ─── 游戏状态 (local_save_data) ──────────────────────────────────────────
  /// 对应 JS saveLocalState() 的完整快照
  static Future<void> saveGameState({
    required List<List<Map<String, dynamic>?>> board,
    required int score,
    required int combo,
    required int bestScore,
    required int maxCombo,
    required int novelIndex,
    String? currentNovelId,
  }) async {
    final st = {
      'board_data': jsonEncode(board),
      'score': score,
      'combo': combo,
      'bestScore': bestScore,
      'maxCombo': maxCombo,
      'novel_index': novelIndex,
      'current_novel_id': currentNovelId,
    };
```

```
    await _p.setString(_kLocalSave, jsonEncode(st));
    await _p.setInt(_kBestScore, bestScore);
    await _p.setInt(_kMaxCombo, maxCombo);
    await _p.setInt(_kNovelIndex, novelIndex);
    if (currentNovelId != null) {
      await _p.setString(_kCurrentNovelId, currentNovelId);
    }
  }
  static Map<String, dynamic>? loadGameState() {
    final saved = _p.getString(_kLocalSave);
    if (saved == null) return null;
    try {
      return jsonDecode(saved) as Map<String, dynamic>;
    } catch (_) {
      return null;
    }
  }
  static int loadBestScore() => _p.getInt(_kBestScore) ?? 0;
  static int loadMaxCombo() => _p.getInt(_kMaxCombo) ?? 0;
  // ─── 书架元数据 (local_bookshelf) ──────────────────────────────────────────
  static Future<void> saveBookshelf(List<Map<String, dynamic>> shelf) async {
    await _p.setString(_kBookshelf, jsonEncode(shelf));
  }
  static List<Map<String, dynamic>> loadBookshelf() {
    final s = _p.getString(_kBookshelf);
    if (s == null) return [];
    try {
      return (jsonDecode(s) as List).cast<Map<String, dynamic>>();
    } catch (_) {
      return [];
    }
  }
  // ─── 阅读进度 (reading_records / ProgressManager) ──────────────────────────
  static Future<void> updateReadingRecord(
    String bookId,
    int cursor,
    int total,
  ) async {
    if (total <= 0) return;
    final records = _loadReadingRecords();
    final percent = (cursor / total * 100).clamp(0.0, 100.0);
    records[bookId] = {
      'cursor': cursor,
      'total': total,
      'percent': double.parse(percent.toStringAsFixed(2)),
    };
    await _p.setString(_kReadingRecords, jsonEncode(records));
  }
  static Map<String, dynamic> getReadingRecord(String bookId) {
    final records = _loadReadingRecords();
```

```
    return records[bookId] as Map<String, dynamic>? ??
        {'cursor': 0, 'total': 1, 'percent': 0.0};
  }
  static Future<void> deleteReadingRecord(String bookId) async {
    final records = _loadReadingRecords();
    records.remove(bookId);
    await _p.setString(_kReadingRecords, jsonEncode(records));
  }
  static Map<String, dynamic> _loadReadingRecords() {
    final s = _p.getString(_kReadingRecords);
    if (s == null) return {};
    try {
      return (jsonDecode(s) as Map).cast<String, dynamic>();
    } catch (_) {
      return {};
    }
  }
  static Future<void> setCurrentNovelId(String? id) async {
    if (id == null) {
      await _p.remove(_kCurrentNovelId);
    } else {
      await _p.setString(_kCurrentNovelId, id);
    }
  }
  static String? getCurrentNovelId() => _p.getString(_kCurrentNovelId);
  static int getCurrentNovelIndex() => _p.getInt(_kNovelIndex) ?? 0;
  static int getCurrentChapterIndex() => _p.getInt(_kCurrentChapterIndex) ?? 0;
  static Future<void> setCurrentChapterIndex(int index) async {
    await _p.setInt(_kCurrentChapterIndex, index);
  }
  static Future<void> setCurrentNovelIndex(int index) async {
    await _p.setInt(_kNovelIndex, index);
  }
  // ── 书籍正文内容（替代 IndexedDB LocalDB.js）──────────────────────────
  static Future<File> _bookFile(String bookId) async {
    final dir = await getApplicationDocumentsDirectory();
    return File('${dir.path}/books/$bookId.json');
  }
  static Future<void> saveBookContent(
    String bookId, {
    required List<String> lines,
    required List<Map<String, dynamic>> chapters,
  }) async {
    try {
      final file = await _bookFile(bookId);
      await file.parent.create(recursive: true);
      await file
          .writeAsString(jsonEncode({'lines': lines, 'chapters': chapters}));
    } catch (e, st) {
      CyberLogger.captureWarning(
```

```
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.saveBookContent 失败'},
      );
    }
  }
  static Future<Map<String, dynamic>?> loadBookContent(String bookId) async {
    try {
      final file = await _bookFile(bookId);
      if (!await file.exists()) return null;
      final content = await file.readAsString();
      if (content.trim().isEmpty) return null;
      return jsonDecode(content) as Map<String, dynamic>;
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.loadBookContent 失败'},
      );
      return null;
    }
  }
  static Future<void> deleteBookContent(String bookId) async {
    try {
      final file = await _bookFile(bookId);
      if (await file.exists()) await file.delete();
    } catch (_) {}
  }
  // ── 全局设置 ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
  static bool getSettingSound() => _p.getBool(_kSettingSound) ?? true;
  static Future<void> setSettingSound(bool v) => _p.setBool(_kSettingSound, v);
  static bool getSettingStoryTts() => _p.getBool(_kSettingStoryTts) ?? true;
  static Future<void> setSettingStoryTts(bool v) =>
      _p.setBool(_kSettingStoryTts, v);
  static String getSettingVoice() =>
      _p.getString(_kSettingVoice) ?? 'zh-CN-XiaoxiaoNeural';
  static Future<void> setSettingVoice(String v) =>
      _p.setString(_kSettingVoice, v);
  static int getSettingIdleTimeout() => _p.getInt(_kSettingIdleTimeout) ?? 1;
  static Future<void> setSettingIdleTimeout(int v) =>
      _p.setInt(_kSettingIdleTimeout, v);
  static double getSettingTtsRate() => _p.getDouble(_kSettingTtsRate) ?? 1.0;
  static Future<void> setSettingTtsRate(double v) =>
      _p.setDouble(_kSettingTtsRate, v);
  static double getSettingAmbientVol() =>
      _p.getDouble(_kSettingAmbientVol) ?? 0.5;
  static Future<void> setSettingAmbientVol(double v) =>
      _p.setDouble(_kSettingAmbientVol, v);
```

```
0244    static bool getSettingAmbientEnabled() =>
0245        _p.getBool(_kSettingAmbientEnabled) ?? true;
0246    static Future<void> setSettingAmbientEnabled(bool v) =>
0247        _p.setBool(_kSettingAmbientEnabled, v);
0249    static String getSettingAmbientStyle() =>
0250        _p.getString(_kSettingAmbientStyle) ?? 'wuxia';
0251    static Future<void> setSettingAmbientStyle(String v) =>
0252        _p.setString(_kSettingAmbientStyle, v);
0254    static String getSettingAnimationQuality() =>
0255        _p.getString(_kSettingAnimationQuality) ?? 'auto';
0256    static Future<void> setSettingAnimationQuality(String v) =>
0257        _p.setString(_kSettingAnimationQuality, v);
0259    // ─── 隐私协议同意状态 ──────────────────────────────────────────────────────────────────────────────────────────────────────────
0260    static bool hasAgreedPrivacy() => _p.getBool(_kHasAgreedPrivacy) ?? false;
0261    static Future<void> setHasAgreedPrivacy(bool v) =>
0262        _p.setBool(_kHasAgreedPrivacy, v);
0264    // ─── 粘性位：用户是否曾主动选择过书籍 ──────────────────────────────────────────────────────────────────────────────────────────
0265    static bool hasSelectedBook() => _p.getBool(_kHasSelectedBook) ?? false;
0266    static Future<void> setHasSelectedBook(bool v) =>
0267        _p.setBool(_kHasSelectedBook, v);
0269    // ─── 分章文本缓存（文件路径，非 SharedPreferences）──────────────────────────────────────────────────────────────
0270    static Future<File> _chapterCacheFile(String bookId, int chapterIndex) async {
0271      final dir = await getApplicationDocumentsDirectory();
0272      final idx = chapterIndex.toString().padLeft(3, '0');
0273      return File('${dir.path}/books/chapters/$bookId/$idx.txt');
0274    }
0276    static Future<Directory> _chapterCacheDir(String bookId) async {
0277      final dir = await getApplicationDocumentsDirectory();
0278      return Directory('${dir.path}/books/chapters/$bookId');
0279    }
0281    static Future<void> saveChapterCache(
0282      String bookId,
0283      int chapterIndex,
0284      String text,
0285    ) async {
0286      try {
0287        final file = await _chapterCacheFile(bookId, chapterIndex);
0288        await file.parent.create(recursive: true);
0289        await file.writeAsString(text);
0290      } catch (e, st) {
0291        CyberLogger.captureWarning(
0292          e,
0293          stack: st,
0294          tag: 'library',
0295          extra: {'context': 'StorageService.saveChapterCache 失败'},
0296        );
0297      }
0298    }
0300    static Future<String?> loadChapterCache(
0301      String bookId,
```

```
    int chapterIndex,
  ) async {
    try {
      final file = await _chapterCacheFile(bookId, chapterIndex);
      if (!await file.exists()) return null;
      final content = await file.readAsString();
      return content.trim().isEmpty ? null : content;
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.loadChapterCache 失败'},
      );
      return null;
    }
  }
  static Future<void> clearChapterCache(String bookId) async {
    try {
      final dir = await _chapterCacheDir(bookId);
      if (await dir.exists()) await dir.delete(recursive: true);
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.clearChapterCache 失败'},
      );
    }
  }
  /// LRU 清理：保留 [currentChapterIndex ± keepAround] 范围内文件，其余删除。
  static Future<void> pruneChapterCache(
    String bookId,
    int currentChapterIndex, {
    int keepAround = 3,
  }) async {
    try {
      final dir = await _chapterCacheDir(bookId);
      if (!await dir.exists()) return;
      final keepMin = currentChapterIndex - keepAround;
      final keepMax = currentChapterIndex + keepAround;
      await for (final entity in dir.list()) {
        if (entity is! File) continue;
        final name = entity.uri.pathSegments.last;
        if (!name.endsWith('.txt')) continue;
        final idx = int.tryParse(name.replaceFirst('.txt', ''));
        if (idx == null) continue;
        if (idx < keepMin || idx > keepMax) await entity.delete();
      }
    } catch (e, st) {
```

```
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.pruneChapterCache 失败'},
      );
    }
  }
  // ─── 书目录缓存（文件路径）──────────────────────────────────────────────────────────────
  static Future<File> _catalogFile(String bookId) async {
    final dir = await getApplicationDocumentsDirectory();
    return File('${dir.path}/books/catalogs/${bookId}_catalog.json');
  }
  static Future<void> saveBookCatalog(
    String bookId,
    List<Map<String, dynamic>> chapters,
  ) async {
    try {
      final file = await _catalogFile(bookId);
      await file.parent.create(recursive: true);
      await file.writeAsString(jsonEncode(chapters));
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.saveBookCatalog 失败'},
      );
    }
  }
  static Future<List<Map<String, dynamic>>?> loadBookCatalog(
    String bookId,
  ) async {
    try {
      final file = await _catalogFile(bookId);
      if (!await file.exists()) return null;
      final content = await file.readAsString();
      if (content.trim().isEmpty) return null;
      return (jsonDecode(content) as List).cast<Map<String, dynamic>>();
    } catch (e, st) {
      CyberLogger.captureWarning(
        e,
        stack: st,
        tag: 'library',
        extra: {'context': 'StorageService.loadBookCatalog 失败'},
      );
      return null;
    }
  }
}
```

```
// ===== 文件：lib/core/theme/cyber_colors.dart =====
0001  import 'package:flutter/material.dart';
0003  class CyberColors {
0004    // 极致的深色背景
0005    static const Color background = Color(0xFF000000); // 真正的高级黑
0007    // 赛博/黑客帝国经典的荧光绿（主色调）
0008    static const Color neonGreen = Color(0xFF00FF41);
0010    // 赛博朋克必不可少的霓虹粉
0011    static const Color neonPink = Color(0xFFFE019A);
0013    // 霓虹紫色
0014    static const Color neonPurple = Color(0xFF8B5CF6);
0016    // 霓虹青色
0017    static const Color neonCyan = Color(0xFF22D3EE);
0019    // 卡片背景色（带一点透明度的深灰，制造玻璃感）
0020    static const Color cardBackground = Color(0xFF13141E);
0021    static const Color surface = Color(0xFF1A1B28);
0023    // 提词器未读文字的暗色
0024    static const Color textDim = Color(0xFF4A5568);
0026    // 毛玻璃深底色（灵动岛 / 顶部工具栏 / 弹窗共用）
0027    static const Color glassDark = Color(0xD90A0A0F);
0029    // 播放按钮渐变粉（CyberPlayerConsole 播放按钮起始色）
0030    static const Color hotPink = Color(0xFFEC4899);
0032    // 章节列表 / 弹窗背景
0033    static const Color panelBackground = Color(0xFF0D0E18);
0035    // 白色语义化透明度
0036    static const Color whiteHigh = Color(0xD9FFFFFF); // 85%
0037    static const Color whiteMedium = Color(0x99FFFFFF); // 60%
0038    static const Color whiteDim = Color(0x8AFFFFFF); // 54%
0039    static const Color whiteMuted = Color(0x61FFFFFF); // 38%
0040    static const Color whiteSubtle = Color(0x3DFFFFFF); // 24%
0041    static const Color whiteFaint = Color(0x14FFFFFF); // 8%
0042    static const Color whiteBorder = Color(0x1AFFFFFF); // 10%
0044    // 黑色语义化透明度（阴影 / 遮罩 / overlay）
0045    static const Color blackOverlay = Color(0xB3000000); // 70%
0046    static const Color blackShadow = Color(0x80000000); // 50%
0047    static const Color blackDim = Color(0x8A000000); // 54%
0049    // 柔和的发光阴影色
0050    static const Color glowShadow = Color(0x8800FF41);
0051    static const Color pinkGlow = Color(0x88FE019A);
0053    // 2048 方块配色 - 提取自旧版style.css的渐变色
0054    static const Color tile2Start = Color(0xFF43e97b);
0055    static const Color tile2End = Color(0xFF38f9d7);
0057    static const Color tile4Start = Color(0xFF00f2fe);
0058    static const Color tile4End = Color(0xFF4facfe);
0060    static const Color tile8Start = Color(0xFF4facfe);
0061    static const Color tile8End = Color(0xFF00c6fb);
0063    static const Color tile16Start = Color(0xFF0093e9);
0064    static const Color tile16End = Color(0xFF80d0c7);
0066    static const Color tile32Start = Color(0xFFa18cd1);
```

```
0067    static const Color tile32End = Color(0xFFfbc2eb);
0069    static const Color tile64Start = Color(0xFFc471f5);
0070    static const Color tile64End = Color(0xFFfa71cd);
0072    static const Color tile128Start = Color(0xFFe040fb);
0073    static const Color tile128End = Color(0xFF8e24aa);
0075    static const Color tile256Start = Color(0xFFf857a6);
0076    static const Color tile256End = Color(0xFFff5858);
0078    static const Color tile512Start = Color(0xFFf7971e);
0079    static const Color tile512End = Color(0xFFffd200);
0081    static const Color tile1024Start = Color(0xFFff6a00);
0082    static const Color tile1024End = Color(0xFFee0979);
0084    static const Color tile2048Start = Color(0xFFf12711);
0085    static const Color tile2048End = Color(0xFFf5af19); // 亮红
0086    static const Color tile512 = Color(0xFFFFD60A); // 金色
0088    // 工具色
0089    static const Color white = Color(0xFFFFFFFF); // 纯白（Canvas 高光专用）
0090    static const Color transparent = Color(0x00000000); // 透明（Material 背景专用）
0091    static const Color tileGold = Color(0xFFFFD700); // 传奇金色（square_board 溯源）
0092    static const Color hackerBlue =
0093        Color(0xFF00F3FF); // 骇客蓝（规范中的 #00f3ff，吹着天 BGM）
0094    static const Color tileBlue = Color(0xFF3B82F6); // 方块低数粒子色（tile_widget 溯源）
0095  }

// ===== 文件：lib/core/theme/cyber_dimensions.dart =====
0001  /// 赛博朋克设计系统 - 尺寸规范
0002  /// 统一管理圆角、边框、模糊、间距等数值，确保视觉一致性
0003  class CyberDimensions {
0004    // ==================== 圆角系统 ====================
0005    // 5 级圆角，从大到小递减
0007    /// 超大圆角 - 用于大型容器（棋盘外框）
0008    static const double radiusXL = 32.0;
0010    /// 大圆角 - 用于中型容器（控制台、工具栏、弹窗）
0011    static const double radiusL = 24.0;
0013    /// 中圆角 - 用于卡片、格子
0014    static const double radiusM = 16.0;
0016    /// 小圆角 - 用于按钮、小卡片
0017    static const double radiusS = 12.0;
0019    /// 超小圆角 - 用于细节（进度条、波形条）
0020    static const double radiusXS = 2.0;
0022    // ==================== 边框系统 ====================
0023    // 3 级边框宽度
0025    /// 粗边框 - 用于强调（选中状态、主要边框）
0026    static const double borderThick = 1.5;
0028    /// 常规边框 - 用于普通容器
0029    static const double borderNormal = 1.0;
0031    /// 细边框 - 用于细微装饰（背景网格）
0032    static const double borderThin = 0.5;
0034    // ==================== 毛玻璃模糊系统 ====================
0035    // 3 级模糊强度
0037    /// 强模糊 - 用于主要容器（棋盘、卡片）
```

```
0038    static const double blurStrong = 20.0;
0040    /// 中模糊 - 用于工具栏、控制台
0041    static const double blurMedium = 15.0;
0043    /// 轻模糊 - 用于遮罩、头部
0044    static const double blurLight = 10.0;
0046    // ==================== 间距系统 ====================
0047    // 标准间距，8px 倍数
0049    static const double spacingXXS = 2.0;
0050    static const double spacingXS = 4.0;
0051    static const double spacingS = 8.0;
0052    static const double spacingMS = 12.0;
0053    static const double spacingM = 16.0;
0054    static const double spacingML = 20.0;
0055    static const double spacingL = 24.0;
0056    static const double spacingXL = 32.0;
0058    // ==================== 头部 / 工具栏 ====================
0060    /// 标准页面头部高度
0061    static const double headerHeight = 56.0;
0063    /// 标准电传屏高度
0064    static const double teleprompterHeight = 46.0;
0066    /// 标准电传屏遮罩宽度
0067    static const double teleprompterMaskWidth = 40.0;
0069    /// 标准仪表盘吉祥物宽度
0070    static const double dashboardMascotWidth = 68.0;
0072    /// 标准仪表盘吉祥物高度
0073    static const double dashboardMascotHeight = 84.0;
0075    /// 标准仪表盘缓冲区高度
0076    static const double dashboardBoardBuffer = 76.0;
0078    /// 标准仪表盘状态卡片最小高度
0079    static const double dashboardStatusCardMinHeight = 85.0;
0081    // ==================== 动效时长 ====================
0083    /// 通用过渡动画时长（Modal、开关、按钮状态切换）
0084    static const Duration animNormal = Duration(milliseconds: 300);
0086    /// 快速反馈动画时长（弹窗退场、选项切换）
0087    static const Duration animFast = Duration(milliseconds: 250);
0089    /// 超快反馈动画时长（棋盘倾斜、元素位移）
0090    static const Duration animXFast = Duration(milliseconds: 150);
0092    /// 瞬间反馈动画时长（眼球追踪、合并弹出）
0093    static const Duration animInstant = Duration(milliseconds: 120);
0095    /// 中等动画时长（合并粒子、导入按钮）
0096    static const Duration animMedium = Duration(milliseconds: 400);
0098    /// 消除动画时长（棋盘方块消除三段式）
0099    static const Duration animEliminate = Duration(milliseconds: 450);
0101    /// 慢动画时长（棋盘重置、身体跳跃）
0102    static const Duration animSlow = Duration(milliseconds: 600);
0104    /// Toast 显示持续时长
0105    static const Duration toastDuration = Duration(seconds: 2, milliseconds: 500);
0107    // ==================== 动效（视觉） ====================
0109    /// 提词器扫光条宽度
0110    static const double shimmerWidth = 80.0;
```

```
0112    /// 发光阴影模糊半径（霓虹光晕）
0113    static const double glowBlurRadius = 8.0;
0115    /// 发光阴影扩散半径
0116    static const double glowSpreadRadius = 1.0;
0118    // ==================== 列表 / 网格 ====================
0120    /// 章节列表 item 固定高度（复用 headerHeight）
0121    static const double chapterItemHeight = headerHeight;
0123    /// 状态圆点尺寸（已读/未读指示器）
0124    static const double statusDotSize = 6.0;
0126    // ==================== 图标尺寸 ====================
0128    /// 超小图标（箭头、指示器）
0129    static const double iconXS = 16.0;
0131    /// 小图标（音量、状态）
0132    static const double iconS = 18.0;
0134    /// 中图标（操作按钮、加载器）
0135    static const double iconM = 20.0;
0137    /// 大图标（对话框标题图标）
0138    static const double iconL = 28.0;
0139  }

// ===== 文件：lib/core/theme/cyber_shadows.dart =====
0001  import 'package:flutter/material.dart';
0002  import 'cyber_colors.dart';
0004  /// 赛博朋克设计系统 - 阴影规范
0005  /// 统一管理阴影效果，确保视觉层次一致
0006  class CyberShadows {
0007    // ==================== 标准阴影系统 ====================
0009    /// 悬浮阴影 - 用于大型容器（棋盘、卡片）
0010    /// 强烈的悬浮感，适合主要内容区域
0011    static const List<BoxShadow> elevated = [
0012      BoxShadow(
0013        color: CyberColors.blackShadow,
0014        blurRadius: 30,
0015        offset: Offset(0, 10),
0016      ),
0017    ];
0019    /// 浮动阴影 - 用于中型组件（书架卡片、导入按钮）
0020    /// 轻微悬浮感，适合次要内容
0021    static const List<BoxShadow> floating = [
0022      BoxShadow(
0023        color: CyberColors.blackShadow,
0024        blurRadius: 16,
0025        offset: Offset(0, 6),
0026      ),
0027    ];
0029    /// 贴近阴影 - 用于小型组件（播放按钮、小卡片）
0030    /// 贴近表面，适合细节装饰
0031    static const List<BoxShadow> subtle = [
0032      BoxShadow(
0033        color: CyberColors.blackShadow,
```

```
      blurRadius: 10,
      offset: Offset(0, 4),
    ),
  ];
  // ==================== 霓虹光晕阴影 ====================
  /// 创建霓虹光晕效果
  /// 用于方块、按钮等需要发光效果的元素
  static List<BoxShadow> neonGlow({
    required Color color,
    double intensity = 1.0,
  }) {
    return [
      BoxShadow(
        color: color.withValues(alpha: 0.6 * intensity),
        blurRadius: 12 * intensity,
        spreadRadius: 2 * intensity,
      ),
      BoxShadow(
        color: color.withValues(alpha: 0.3 * intensity),
        blurRadius: 18 * intensity,
        spreadRadius: 0,
      ),
    ];
  }
}

// ===== 文件：lib/core/theme/cyber_text_styles.dart =====
import 'package:flutter/material.dart';
import 'cyber_colors.dart';
class CyberTextStyles {
  static const String monoFont = 'JetBrains Mono';
  // 提词器正在阅读的【高亮发光文字】
  static const TextStyle teleprompterActive = TextStyle(
    color: CyberColors.neonGreen,
    fontSize: 24,
    fontWeight: FontWeight.bold,
    letterSpacing: 2.0, // 字间距拉开，更有极客敲代码的呼吸感
    shadows: [
      Shadow(
        blurRadius: 10.0, // 发光半径
        color: CyberColors.glowShadow,
        offset: Offset(0, 0),
      ),
    ],
  );
  // 提词器下方还没读到的【暗色背景文字】
  static const TextStyle teleprompterDim = TextStyle(
    color: CyberColors.textDim,
    fontSize: 24,
    fontWeight: FontWeight.normal,
```

```
0026      letterSpacing: 2.0,
0027    );
0029    // 2048 棋盘上的数字字体
0030    static const TextStyle gameGridNumber = TextStyle(
0031      color: CyberColors.background,
0032      fontSize: 28,
0033      fontWeight: FontWeight.w900,
0034    );
0036    // ==================== 通用 UI 文本样式 ====================
0038    /// 页面头部标题（18px, bold, letterSpacing: 2）
0039    /// 使用时通过 copyWith(color: ...) 设置具体颜色
0040    static const TextStyle screenTitle = TextStyle(
0041      fontSize: 18,
0042      fontWeight: FontWeight.bold,
0043      letterSpacing: 2,
0044    );
0046    /// 分区标题（12px, bold, letterSpacing: 1.5, neonGreen）
0047    static const TextStyle sectionLabel = TextStyle(
0048      color: CyberColors.neonGreen,
0049      fontSize: 12,
0050      fontWeight: FontWeight.bold,
0051      letterSpacing: 1.5,
0052    );
0054    /// 列表项主标题（14px, whiteHigh）
0055    static const TextStyle tileTitle = TextStyle(
0056      color: CyberColors.whiteHigh,
0057      fontSize: 14,
0058    );
0060    /// 列表项副标题（12px, whiteMuted）
0061    static const TextStyle tileSubtitle = TextStyle(
0062      color: CyberColors.whiteMuted,
0063      fontSize: 12,
0064    );
0066    /// 标签文字（14px, whiteDim）
0067    static const TextStyle labelMedium = TextStyle(
0068      color: CyberColors.whiteDim,
0069      fontSize: 14,
0070    );
0072    /// 小号正文（13px, whiteDim）—— 下拉菜单、标签芯片等
0073    static const TextStyle bodySmall = TextStyle(
0074      color: CyberColors.whiteDim,
0075      fontSize: 13,
0076    );
0078    /// 小号正文加粗（13px, w600, whiteHigh）
0079    static const TextStyle bodySmallBold = TextStyle(
0080      color: CyberColors.whiteHigh,
0081      fontSize: 13,
0082      fontWeight: FontWeight.w600,
0083    );
0085    static const TextStyle overlineTiny = TextStyle(
```

```
0086      color: CyberColors.whiteDim,
0087      fontSize: 9,
0088      fontWeight: FontWeight.w700,
0089      letterSpacing: 0.5,
0090    );
0092    static const TextStyle segmentLabel = TextStyle(
0093      color: CyberColors.whiteHigh,
0094      fontSize: 14,
0095      fontWeight: FontWeight.w600,
0096      letterSpacing: 0.3,
0097    );
0099    static const TextStyle teleprompterInlineRead = TextStyle(
0100      color: CyberColors.neonCyan,
0101      fontSize: 18,
0102      fontWeight: FontWeight.w800,
0103      letterSpacing: 0.5,
0104      height: 1.0,
0105    );
0107    static const TextStyle teleprompterInlineUnread = TextStyle(
0108      color: CyberColors.whiteMuted,
0109      fontSize: 18,
0110      fontWeight: FontWeight.w500,
0111      letterSpacing: 0.5,
0112      height: 1.0,
0113    );
0115    static const TextStyle teleprompterError = TextStyle(
0116      color: CyberColors.whiteHigh,
0117      fontSize: 12,
0118      fontWeight: FontWeight.w700,
0119    );
0121    /// 提词器浮层小号错误标题（10px bold）
0122    static const TextStyle teleprompterErrorTitle = TextStyle(
0123      fontSize: 10,
0124      fontWeight: FontWeight.bold,
0125    );
0127    /// 提词器浮层小号提示文字（9px）
0128    static const TextStyle teleprompterErrorHint = TextStyle(
0129      fontSize: 9,
0130    );
0132    static const TextStyle teleprompterPlaceholder = TextStyle(
0133      color: CyberColors.whiteMuted,
0134      fontSize: 14,
0135      fontWeight: FontWeight.w600,
0136    );
0138    static const TextStyle dashboardCounter = TextStyle(
0139      color: CyberColors.neonCyan,
0140      fontSize: 22,
0141      fontFamily: monoFont,
0142      fontWeight: FontWeight.w900,
0143    );
```

```
  static const TextStyle dashboardSeparator = TextStyle(
    color: CyberColors.whiteSubtle,
    fontSize: 16,
    fontFamily: monoFont,
    fontWeight: FontWeight.w300,
  );
  static const TextStyle captionBold = TextStyle(
    color: CyberColors.whiteDim,
    fontSize: 12,
    fontWeight: FontWeight.bold,
  );
  static const TextStyle captionTight = TextStyle(
    color: CyberColors.whiteDim,
    fontSize: 12,
    height: 1.3,
  );
  static const TextStyle captionComfortable = TextStyle(
    color: CyberColors.whiteDim,
    fontSize: 12,
    height: 1.4,
  );
  static const TextStyle captionHint = TextStyle(
    color: CyberColors.whiteDim,
    fontSize: 11,
    height: 1.5,
  );
  /// 提示/说明文字（12px, whiteDim）
  static const TextStyle caption = TextStyle(
    color: CyberColors.whiteDim,
    fontSize: 12,
  );
  /// 对话框标题（18px, bold）
  /// 使用时通过 copyWith(color: ...) 设置具体颜色
  static const TextStyle dialogTitle = TextStyle(
    fontSize: 18,
    fontWeight: FontWeight.bold,
  );
  /// 按钮文字（14px, bold）
  static const TextStyle buttonLabel = TextStyle(
    fontSize: 14,
    fontWeight: FontWeight.bold,
  );
}

// ===== 文件：lib/core/utils/cyber_logger.dart =====
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:sentry_flutter/sentry_flutter.dart';
/// 赛博朋克风格全局错误捕获、日志格式化与 Sentry 分级上报模块
///
```

```
      child: Row(
        children: options.entries.map((entry) {
          final isSelected = entry.key == value;
          return Expanded(
            child: GestureDetector(
              onTap: () => onChanged(entry.key),
              child: AnimatedContainer(
                duration: CyberDimensions.animFast,
                padding: const EdgeInsets.symmetric(vertical: 10),
                decoration: BoxDecoration(
                  gradient: isSelected
                      ? LinearGradient(
                          begin: Alignment.topLeft,
                          end: Alignment.bottomRight,
                          colors: [
                            CyberColors.whiteFaint.withValues(alpha: 0.2),
                            CyberColors.whiteFaint.withValues(alpha: 0.05),
                          ],
                        )
                      : null,
                  borderRadius: BorderRadius.circular(CyberDimensions.radiusL),
                  boxShadow: isSelected
                      ? [
                          BoxShadow(
                            color:
                                CyberColors.background.withValues(alpha: 0.4),
                            blurRadius: 4,
                            offset: const Offset(0, 2),
                          ),
                        ]
                      : null,
                ),
                child: Center(
                  child: Text(
                    entry.value,
                    style: CyberTextStyles.bodySmall.copyWith(
                      color: isSelected
                          ? CyberColors.white
                          : CyberColors.whiteMuted,
                      fontWeight:
                          isSelected ? FontWeight.bold : FontWeight.normal,
                      fontSize: 12,
                    ),
                  ),
                ),
              ),
            ),
          );
        }).toList(),
      ),
```

```
    );
  }
}
class _VoiceSelector extends ConsumerWidget {
  final SettingsProvider settings;
  static const Map<String, String> _voices = {
    'zh-CN-XiaoxiaoNeural': '晓晓 (温柔)',
    'zh-CN-YunxiNeural': '云溪 (阳光)',
    'zh-CN-YunjianNeural': '云健 (稳重)',
    'zh-CN-XiaoyiNeural': '晓伊 (知性)',
    'zh-CN-XiaomengNeural': '晓梦 (活泼)',
  };
  const _VoiceSelector({required this.settings});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: CyberDimensions.spacingM),
      decoration: BoxDecoration(
        color: CyberColors.surface,
        borderRadius: BorderRadius.circular(CyberDimensions.radiusM),
        border: Border.all(color: CyberColors.whiteFaint),
      ),
      child: DropdownButton<String>(
        value: _voices.containsKey(settings.voice) ? settings.voice : null,
        isExpanded: true,
        dropdownColor: CyberColors.surface,
        underline: const SizedBox.shrink(),
        icon: const Icon(
          Icons.keyboard_arrow_down,
          color: CyberColors.whiteMuted,
        ),
        items: _voices.entries.map((e) {
          return DropdownMenuItem(
            value: e.key,
            child: Text(
              e.value,
              style: CyberTextStyles.bodySmall.copyWith(
                color: CyberColors.whiteDim,
                fontSize: 13,
              ),
            ),
          );
        }).toList(),
        onChanged: (v) async {
          if (v == null) return;
          await settings.setVoice(v);
          if (settings.storyTts) {
            ref.read(ttsAudioProvider.notifier).refreshSession();
          }
        },
```

```
      ),
    );
  }
}
class _TtsTestButton extends ConsumerStatefulWidget {
  const _TtsTestButton();
  @override
  ConsumerState<_TtsTestButton> createState() => _TtsTestButtonState();
}
class _TtsTestButtonState extends ConsumerState<_TtsTestButton> {
  bool _isTesting = false;
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: _isTesting ? CyberColors.surface : CyberColors.transparent,
        borderRadius: BorderRadius.circular(CyberDimensions.radiusM),
        border: Border.all(
          color: CyberColors.neonCyan.withValues(alpha: 0.3),
        ),
      ),
      child: Material(
        color: CyberColors.transparent,
        child: InkWell(
          onTap: _isTesting ? null : _testTtsConnection,
          borderRadius: BorderRadius.circular(CyberDimensions.radiusM),
          child: Padding(
            padding: const EdgeInsets.symmetric(
              horizontal: CyberDimensions.spacingM,
              vertical: CyberDimensions.spacingMS,
            ),
            child: Row(
              children: [
                if (_isTesting)
                  const SizedBox(
                    width: 12,
                    height: 12,
                    child: CircularProgressIndicator(
                      color: CyberColors.neonCyan,
                      strokeWidth: 2,
                    ),
                  )
                else
                  const Icon(
                    Icons.terminal,
                    color: CyberColors.neonCyan,
                    size: 14,
                  ),
                const SizedBox(width: 12),
                Expanded(
```

```
                    child: Text(
                      _isTesting ? '正在进行神经链路诊断...' : '执行系统自检 (TTS Test)',
                      style: CyberTextStyles.bodySmallBold.copyWith(
                        color: CyberColors.neonCyan,
                        fontSize: 12,
                        letterSpacing: 0.5,
                      ),
                    ),
                  ),
                  const Icon(
                    Icons.chevron_right,
                    color: CyberColors.whiteMuted,
                    size: 14,
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
  Future<void> _testTtsConnection() async {
    final tts = ref.read(ttsEngineProvider);
    setState(() => _isTesting = true);
    try {
      final result = await tts.testConnection();
      if (!mounted) return;
      setState(() => _isTesting = false);
      showDialog(
        context: context,
        builder: (context) => _TtsTestResultDialog(result: result),
      );
    } catch (e, st) {
      CyberLogger.captureWarning(
        e is Exception ? e : Exception('$e'),
        stack: st,
        tag: 'tts',
        extra: {'context': 'TTS 自检触发异常'},
      );
      if (!mounted) return;
      setState(() => _isTesting = false);
      CyberToast.show(
        CyberErrorMessages.testFailedUnresponsive,
        type: ToastType.error,
      );
    }
  }
}
class _TtsTestResultDialog extends StatelessWidget {
  final Map<String, dynamic> result;
```

```
const _TtsTestResultDialog({required this.result});

@override
Widget build(BuildContext context) {
  final success = result['success'] as bool;
  final steps = result['steps'] as List<Map<String, dynamic>>;
  return Dialog(
    backgroundColor: CyberColors.panelBackground,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(CyberDimensions.radiusL),
      side: BorderSide(
        color: success ? CyberColors.neonGreen : CyberColors.neonPink,
        width: 1,
      ),
    ),
    child: Padding(
      padding: const EdgeInsets.all(CyberDimensions.spacingML),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            success ? '链路诊断报告：通畅' : '链路诊断报告：故障',
            style: CyberTextStyles.dialogTitle.copyWith(
              color: success ? CyberColors.neonGreen : CyberColors.neonPink,
              fontSize: 16,
            ),
          ),
          const SizedBox(height: CyberDimensions.spacingM),
          ...steps.take(4).map(
                (step) => Padding(
                  padding: const EdgeInsets.only(bottom: 8.0),
                  child: Row(
                    children: [
                      Icon(
                        step['status'] == 'success'
                            ? Icons.check
                            : Icons.close,
                        size: 14,
                        color: step['status'] == 'success'
                            ? CyberColors.neonGreen
                            : CyberColors.neonPink,
                      ),
                      const SizedBox(width: 8),
                      Text(
                        step['name'],
                        style: CyberTextStyles.bodySmall.copyWith(
                          color: CyberColors.whiteDim,
                          fontSize: 12,
                        ),
                      ),
```

```
                    ],
                  ),
                ),
              ),
              const SizedBox(height: CyberDimensions.spacingML),
              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: () => Navigator.of(context).pop(),
                  style: ElevatedButton.styleFrom(
                    backgroundColor:
                        success ? CyberColors.neonGreen : CyberColors.neonPink,
                    foregroundColor: CyberColors.background,
                    shape: RoundedRectangleBorder(
                      borderRadius:
                          BorderRadius.circular(CyberDimensions.radiusS),
                    ),
                  ),
                  child: const Text('了解', style: CyberTextStyles.buttonLabel),
                ),
              ),
            ],
          ),
        ),
      );
  }
}
class _AmbientVolumeSlider extends StatelessWidget {
  final SettingsProvider settings;
  const _AmbientVolumeSlider({required this.settings});
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: CyberDimensions.spacingS),
      decoration: BoxDecoration(
        color: CyberColors.surface,
        borderRadius: BorderRadius.circular(CyberDimensions.radiusM),
        border: Border.all(color: CyberColors.whiteFaint),
      ),
      child: Row(
        children: [
          const SizedBox(width: 12),
          const Icon(
            Icons.volume_down,
            size: 14,
            color: CyberColors.whiteMuted,
          ),
          Expanded(
            child: SliderTheme(
              data: SliderTheme.of(context).copyWith(
```

```
                activeTrackColor: CyberColors.neonPurple,
                inactiveTrackColor: CyberColors.whiteFaint,
                thumbColor: CyberColors.white,
                overlayColor: CyberColors.neonPurple.withValues(alpha: 0.1),
                trackHeight: 2.0,
                thumbShape: const RoundSliderThumbShape(
                  enabledThumbRadius: 6,
                ),
              ),
              child: Slider(
                value: settings.ambientVol,
                onChanged: (v) async {
                  await settings.setAmbientVol(v);
                  await AmbientService.setVolume(v);
                },
              ),
            ),
          ),
          Text(
            '${(settings.ambientVol * 100).round()}%',
            style: CyberTextStyles.captionBold.copyWith(
              color: CyberColors.neonPurple,
              fontSize: 10,
              fontFamily: CyberTextStyles.monoFont,
            ),
          ),
          const SizedBox(width: 12),
        ],
      ),
    );
  }
}

// ===== 文件：lib/features/settings/presentation/widgets/privacy_agreement_modal.dart =====
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:url_launcher/url_launcher.dart';
import 'package:yueyou/core/theme/cyber_colors.dart';
import 'package:yueyou/core/theme/cyber_dimensions.dart';
import 'package:yueyou/core/theme/cyber_text_styles.dart';
import 'package:yueyou/shared/widgets/cyber_modal.dart';

/// 赛博朋克风格隐私协议弹窗
/// - barrierDismissible: false，强制用户作出选择
/// - 返回 true 表示同意，用户拒绝时直接退出应用
Future<bool> showPrivacyAgreementModal(BuildContext context) async {
  final result = await showCyberModal<bool>(
    context: context,
    barrierDismissible: false,
    child: const _PrivacyAgreementContent(),
  );
```

```
  return result ?? false;
}
class _PrivacyAgreementContent extends StatelessWidget {
  const _PrivacyAgreementContent();
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(CyberDimensions.spacingL),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          // ——— 图标 + 标题 ————————————————————————————————————————————————
          const Icon(
            Icons.security_rounded,
            color: CyberColors.neonCyan,
            size: CyberDimensions.iconL,
          ),
          const SizedBox(height: CyberDimensions.spacingS),
          Text(
            '阅游 · 隐私政策',
            style: CyberTextStyles.dialogTitle.copyWith(
              color: CyberColors.neonCyan,
            ),
          ),
          const SizedBox(height: CyberDimensions.spacingXS),
          Text(
            '神经接驳协议  ·  NEURAL LINK  v1.0',
            style: CyberTextStyles.caption.copyWith(
              color: CyberColors.whiteMuted,
              letterSpacing: 1.5,
            ),
          ),
          const SizedBox(height: CyberDimensions.spacingM),
          // ——— 协议正文（可滚动）————————————————————————————————————————
          Container(
            constraints: BoxConstraints(
              maxHeight: MediaQuery.sizeOf(context).height * 0.4,
            ),
            decoration: BoxDecoration(
              color: CyberColors.whiteFaint,
              borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
              border: Border.all(
                color: CyberColors.neonCyan.withValues(alpha: 0.2),
                width: CyberDimensions.borderThin,
              ),
            ),
            child: const SingleChildScrollView(
              padding: EdgeInsets.all(CyberDimensions.spacingM),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
```

```
        children: [
          _PolicySection(
            icon: '',
            title: '数据存储',
            body: '所有阅读进度、游戏数据及用户设置仅存储于您的本地设备。'
                '我们不会将您的个人数据上传至任何远端服务器。',
          ),
          SizedBox(height: CyberDimensions.spacingM),
          _PolicySection(
            icon: '',
            title: '云端 TTS 合成',
            body: '当您启用听书功能时，当前朗读的文本段落将通过加密信道发送至'
                '云端 TTS 服务以合成音频。除朗读文本外，不传输任何其他信息。',
          ),
          SizedBox(height: CyberDimensions.spacingM),
          _PolicySection(
            icon: '',
            title: '存储权限',
            body: '导入数据芯片（TXT 文件）时，本应用需要访问您设备的本地存储。'
                '此权限仅用于读取您主动选择的文件，不会扫描其他目录。',
          ),
          SizedBox(height: CyberDimensions.spacingM),
          _PolicySection(
            icon: '',
            title: '隐私承诺',
            body: '- 不采集设备标识符或位置信息\n'
                '- 不追踪用户行为或使用习惯\n'
                '- 不向任何第三方共享数据\n'
                '- 符合《个人信息保护法》及 GDPR',
          ),
          SizedBox(height: CyberDimensions.spacingM),
          _PolicySection(
            icon: '',
            title: '开发者信息',
            body: '运营主体：HCL Studio\n'
                '联系邮箱：support@hclstudio.cn\n'
                '隐私问题反馈、数据删除请求请发送邮件至上述邮箱。',
          ),
        ],
      ),
    ),
  ),
  const SizedBox(height: CyberDimensions.spacingM),
  // ─── 提示 + 隐私政策外链 ──────────────────────────────────────────
  Wrap(
    alignment: WrapAlignment.center,
    crossAxisAlignment: WrapCrossAlignment.center,
    children: [
      Text(
        '同意后，您可在设置页面随时重新查看本协议 ',
```

```
                textAlign: TextAlign.center,
                style: CyberTextStyles.caption.copyWith(
                  color: CyberColors.whiteMuted,
                ),
              ),
              GestureDetector(
                onTap: () => launchUrl(
                  Uri.parse(
                    const String.fromEnvironment(
                      'PRIVACY_POLICY_URL',
                      defaultValue: 'https://docs.qq.com/doc/DVXpHSW9qRkFZVVlN',
                    ),
                  ),
                  mode: LaunchMode.externalApplication,
                ),
                child: Text(
                  '阅读完整版《阅游隐私政策》',
                  style: CyberTextStyles.caption.copyWith(
                    color: CyberColors.neonCyan,
                    decoration: TextDecoration.underline,
                    decorationColor: CyberColors.neonCyan,
                  ),
                ),
              ),
            ],
          ),
          const SizedBox(height: CyberDimensions.spacingL),
          // ─── 按钮行 ──────────────────────────────────────────────────────────────
          Row(
            children: [
              // 不同意并退出（危险操作，用粉色/红色边框区分）
              Expanded(
                child: _DeclineButton(
                  onTap: () {
                    Navigator.of(context).pop(false);
                    SystemNavigator.pop();
                  },
                ),
              ),
              const SizedBox(width: CyberDimensions.spacingMS),
              // 同意（主按钮）
              Expanded(
                child: _AgreeButton(
                  onTap: () => Navigator.of(context).pop(true),
                ),
              ),
            ],
          ),
        ],
      ),
```

```
    );
  }
}
// ─── 协议条款单节 ──────────────────────────────────────────────────────────────
class _PolicySection extends StatelessWidget {
  final String icon;
  final String title;
  final String body;
  const _PolicySection({
    required this.icon,
    required this.title,
    required this.body,
  });
  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Text(
              icon,
              style: CyberTextStyles.bodySmall.copyWith(
                fontSize: 14,
              ),
            ),
            const SizedBox(width: CyberDimensions.spacingXS),
            Text(
              title,
              style: CyberTextStyles.bodySmallBold.copyWith(
                color: CyberColors.neonCyan,
              ),
            ),
          ],
        ),
        const SizedBox(height: CyberDimensions.spacingXS),
        Text(
          body,
          style: CyberTextStyles.captionComfortable.copyWith(
            color: CyberColors.whiteMedium,
          ),
        ),
      ],
    );
  }
}
// ─── 拒绝按钮（霓虹粉边框，危险语义）──────────────────────────────────────────
class _DeclineButton extends StatelessWidget {
  final VoidCallback onTap;
  const _DeclineButton({required this.onTap});
```

```
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding:
            const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
        decoration: BoxDecoration(
          color: CyberColors.neonPink.withValues(alpha: 0.08),
          borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
          border: Border.all(
            color: CyberColors.neonPink.withValues(alpha: 0.5),
            width: CyberDimensions.borderNormal,
          ),
        ),
        child: Center(
          child: Text(
            '不同意并退出',
            style: CyberTextStyles.buttonLabel.copyWith(
              color: CyberColors.neonPink,
            ),
          ),
        ),
      ),
    );
  }
}
// ——— 同意按钮（霓虹青渐变，主语义）———————————————————————————————————————
class _AgreeButton extends StatelessWidget {
  final VoidCallback onTap;
  const _AgreeButton({required this.onTap});
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding:
            const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
        decoration: BoxDecoration(
          gradient: const LinearGradient(
            colors: [CyberColors.neonCyan, CyberColors.neonPurple],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          ),
          borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
        ),
        child: Center(
          child: Text(
            '同意',
            style: CyberTextStyles.buttonLabel.copyWith(
```

```
                color: CyberColors.background,
                fontWeight: FontWeight.w900,
              ),
            ),
          ),
        ),
      );
    }
}

// ===== 文件：lib/features/settings/providers/settings_provider.dart =====
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../../core/database/storage_service.dart';
import '../../../core/utils/cyber_performance_detector.dart';
import '../../audio/services/ambient_service.dart';
/// 供 Riverpod 使用的全局设置 Provider
final settingsProvider = ChangeNotifierProvider<SettingsProvider>((ref) {
  final p = SettingsProvider();
  p.loadFromStorage();
  return p;
});
/// 全局设置 Provider
/// 完整复刻旧版 main.js 的 `t` 设置对象 + localStorage 持久化逻辑
/// 键名与 JS setting_* 前缀完全对齐
class SettingsProvider with ChangeNotifier {
  late bool sound;
  late bool storyTts;
  late String voice;
  late int idleTimeout;
  late double ttsRate;
  late double ambientVol;
  late String ambientStyle;
  /// 环境音开关 — 预留字段，待环境音播放器功能实现后启用
  /// 当前无 UI 入口，也无消费方，仅持久化存档保持向后兼容
  late bool ambientEnabled;
  /// 动画质量设置：'auto' | 'high' | 'medium' | 'low'
  late String animationQualitySetting;
  CyberAnimationLevel? _autoDetectedLevel;
  /// App 启动时从 StorageService 恢复所有设置
  void loadFromStorage() {
    sound = StorageService.getSettingSound();
    storyTts = StorageService.getSettingStoryTts();
    voice = StorageService.getSettingVoice();
    // 校验音色有效性，无效则回退默认（防止 storage 残留无效音色名）
    if (voice != 'zh-CN-XiaoxiaoNeural' &&
        voice != 'zh-CN-YunxiNeural' &&
        voice != 'zh-CN-YunjianNeural' &&
        voice != 'zh-CN-XiaoyiNeural' &&
        voice != 'zh-CN-XiaomengNeural') {
```

```
      voice = 'zh-CN-XiaoxiaoNeural';
    }
    idleTimeout = StorageService.getSettingIdleTimeout();
    ttsRate = StorageService.getSettingTtsRate();
    ambientVol = StorageService.getSettingAmbientVol();
    ambientEnabled = StorageService.getSettingAmbientEnabled();
    ambientStyle = StorageService.getSettingAmbientStyle();
    AmbientService.setStyle(ambientStyle);
    animationQualitySetting = StorageService.getSettingAnimationQuality();
    if (animationQualitySetting == 'auto') {
      _autoDetectedLevel = CyberPerformanceDetector.detectLevel();
    }
    notifyListeners();
  }
  /// 计算属性：获取当前界面应当采用的动画等级
  CyberAnimationLevel get currentAnimationLevel {
    switch (animationQualitySetting) {
      case 'high':
        return CyberAnimationLevel.high;
      case 'medium':
        return CyberAnimationLevel.medium;
      case 'low':
        return CyberAnimationLevel.low;
      case 'auto':
      default:
        return _autoDetectedLevel ??= CyberPerformanceDetector.detectLevel();
    }
  }
  Future<void> setSound(bool v) async {
    sound = v;
    await StorageService.setSettingSound(v);
    notifyListeners();
  }
  Future<void> setStoryTts(bool v) async {
    storyTts = v;
    await StorageService.setSettingStoryTts(v);
    notifyListeners();
  }
  Future<void> setVoice(String v) async {
    if (v != 'zh-CN-XiaoxiaoNeural' &&
        v != 'zh-CN-YunxiNeural' &&
        v != 'zh-CN-YunjianNeural' &&
        v != 'zh-CN-XiaoyiNeural' &&
        v != 'zh-CN-XiaomengNeural') {
      v = 'zh-CN-XiaoxiaoNeural';
    }
    voice = v;
    await StorageService.setSettingVoice(v);
    notifyListeners();
  }
```

```
  Future<void> setIdleTimeout(int v) async {
    idleTimeout = v;
    await StorageService.setSettingIdleTimeout(v);
    notifyListeners();
  }
  Future<void> setTtsRate(double v) async {
    ttsRate = v;
    await StorageService.setSettingTtsRate(v);
    notifyListeners();
  }
  Future<void> setAmbientVol(double v) async {
    ambientVol = v;
    await StorageService.setSettingAmbientVol(v);
    notifyListeners();
  }
  Future<void> setAmbientEnabled(bool v) async {
    ambientEnabled = v;
    await StorageService.setSettingAmbientEnabled(v);
    notifyListeners();
  }
  Future<void> setAmbientStyle(String v) async {
    ambientStyle = v;
    await StorageService.setSettingAmbientStyle(v);
    notifyListeners();
  }
  Future<void> setAnimationQualitySetting(String v) async {
    animationQualitySetting = v;
    if (v == 'auto') {
      _autoDetectedLevel = CyberPerformanceDetector.detectLevel();
    }
    await StorageService.setSettingAnimationQuality(v);
    notifyListeners();
  }
}

// ===== 文件：lib/features/update/domain/update_info.dart =====
import 'package:flutter/foundation.dart';
/// 服务端返回的版本信息数据类
///
/// 对应 API 响应格式：
/// ```json
/// {
///   "version": "1.2.0",
///   "buildNumber": 3,
///   "forceUpdate": false,
///   "releaseNotes": "修复了 TTS 连接偶发断线问题",
///   "downloadUrl": "https://..."
/// }
/// ```
@immutable
```

```
class UpdateInfo {
  /// 服务端最新版本号（语义化版本，如 "1.2.0"）
  final String version;
  /// 服务端最新构建号（整数，对应 buildNumber）
  final int buildNumber;
  /// 是否强制更新（true 时禁用跳过按钮）
  final bool forceUpdate;
  /// 版本更新说明（可选，用于展示给用户）
  final String releaseNotes;
  /// Android APK 下载链接（跳转应用市场或自定义页）
  final String downloadUrl;
  const UpdateInfo({
    required this.version,
    required this.buildNumber,
    required this.forceUpdate,
    this.releaseNotes = '',
    this.downloadUrl = '',
  });
  /// 从 JSON 解析（宽松策略：字段缺失时使用安全默认值，不抛异常）
  factory UpdateInfo.fromJson(Map<String, dynamic> json) {
    return UpdateInfo(
      version: json['version'] as String? ?? '0.0.0',
      buildNumber: (json['buildNumber'] as num?)?.toInt() ?? 0,
      forceUpdate: json['forceUpdate'] as bool? ?? false,
      releaseNotes: json['releaseNotes'] as String? ?? '',
      downloadUrl: json['downloadUrl'] as String? ?? '',
    );
  }
  @override
  String toString() =>
      'UpdateInfo(version=$version, buildNumber=$buildNumber, '
      'forceUpdate=$forceUpdate)';
  @override
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is UpdateInfo &&
          version == other.version &&
          buildNumber == other.buildNumber &&
          forceUpdate == other.forceUpdate;
  @override
  int get hashCode => Object.hash(version, buildNumber, forceUpdate);
}

// ===== 文件：lib/features/update/services/update_service.dart =====
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:package_info_plus/package_info_plus.dart';
import 'package:yueyou/core/utils/cyber_logger.dart';
import '../domain/update_info.dart';
/// 热更新版本检测服务
```

```
///
/// ## 工作流程
/// 1. 通过 `PackageInfo.fromPlatform()` 获取本地版本信息
/// 2. 向 `_versionApi` 发送 GET 请求，获取服务端最新版本
/// 3. 对比本地与服务端的 buildNumber（更精确，不受多位语义版本影响）
/// 4. 若服务端版本较新，返回 [UpdateInfo]；否则返回 null
///
/// ## 版本对比策略
/// 优先以 `buildNumber` 整数对比（简单可靠）；
/// 若服务端响应无 `buildNumber`，退级为语义版本字符串对比。
///
/// ## API 接口约定
/// - 请求：`GET https://<host>/version`（无鉴权，公开端点）
/// - 响应 200 + JSON：
///   ```json
///   {"version":"1.2.0","buildNumber":3,"forceUpdate":false,
///    "releaseNotes":"...","downloadUrl":"https://..."}
///   ```
/// - 超时：5 秒（网络异常时静默降级，不中断用户）
///
/// ## 注入方式
/// API URL 通过 `--dart-define=UPDATE_API_URL=https://...` 注入，
/// 默认为空字符串（开发环境不发请求）。
class UpdateService {
  UpdateService._();
  /// 编译时注入的版本检测 API URL
  static const String _versionApi = String.fromEnvironment(
    'UPDATE_API_URL',
    defaultValue: '',
  );
  /// 版本请求超时时间
  static const Duration _timeout = Duration(seconds: 5);
  // ── 公开接口 ────────────────────────────────────────────────────────────
  /// 检测是否有可用更新
  ///
  /// - 返回 [UpdateInfo]：存在更新（版本号大于本地）
  /// - 返回 `null`：无更新、API 未配置或请求失败（静默降级）
  static Future<UpdateInfo?> checkForUpdate() async {
    if (_versionApi.isEmpty) {
      return null;
    }
    try {
      final response = await http.get(Uri.parse(_versionApi)).timeout(_timeout);
      if (response.statusCode != 200) {
        CyberLogger.captureWarning(
          StateError('版本检测接口返回异常'),
          tag: 'update',
          extra: {
            'context': '版本检测 HTTP 响应',
            'statusCode': response.statusCode.toString(),
```

```
        },
      );
      return null;
    }
    final Map<String, dynamic> json =
        jsonDecode(response.body) as Map<String, dynamic>;
    final remote = UpdateInfo.fromJson(json);
    final local = await _getLocalVersion();
    if (_isNewerVersion(remote, local)) {
      CyberLogger.captureMessage(
        '发现新版本: remote=${remote.version}, local=${local.version}',
      );
      return remote;
    }
    CyberLogger.captureMessage('当前已是最新版本: ${local.version}');
    return null;
  } on Exception catch (e, stack) {
    // 网络异常、JSON 解析异常均静默降级，不影响用户体验
    CyberLogger.captureWarning(
      e,
      stack: stack,
      tag: 'update',
      extra: {'context': '版本检测异常，已静默降级'},
    );
    return null;
  }
}
// ─── 内部工具 ──────────────────────────────────────────────────────────────────
/// 获取本地安装版本信息（仅读取，无副作用）
static Future<UpdateInfo> _getLocalVersion() async {
  final info = await PackageInfo.fromPlatform();
  final buildNum = int.tryParse(info.buildNumber) ?? 0;
  return UpdateInfo(
    version: info.version,
    buildNumber: buildNum,
    forceUpdate: false,
  );
}
/// 判断 [remote] 版本是否比 [local] 更新
///
/// 优先比较 buildNumber（整数，精确）；
/// buildNumber 均为 0 时退级为语义版本字符串比较。
static bool isNewerVersion(UpdateInfo remote, UpdateInfo local) =>
    _isNewerVersion(remote, local);
static bool _isNewerVersion(UpdateInfo remote, UpdateInfo local) {
  // 以 buildNumber 优先对比
  if (remote.buildNumber > 0 || local.buildNumber > 0) {
    return remote.buildNumber > local.buildNumber;
  }
  // 退级：语义版本字符串对比（三段 int 比较）
```

```
    return _compareSemanticVersion(remote.version, local.version) > 0;
  }
  /// 语义版本比较，返回：正数 = a 更新，0 = 相同，负数 = b 更新
  static int compareSemanticVersion(String a, String b) =>
      _compareSemanticVersion(a, b);
  static int _compareSemanticVersion(String a, String b) {
    final aParts = _parseSemVer(a);
    final bParts = _parseSemVer(b);
    for (int i = 0; i < 3; i++) {
      final diff = aParts[i] - bParts[i];
      if (diff != 0) return diff;
    }
    return 0;
  }
  /// 将语义版本字符串拆分为三段整数列表（不足三段补 0）
  static List<int> _parseSemVer(String version) {
    final parts = version.split('.');
    return List.generate(3, (i) {
      if (i >= parts.length) return 0;
      return int.tryParse(parts[i]) ?? 0;
    });
  }
}

// ===== 文件：lib/shared/widgets/cyber_confirm_dialog.dart =====
import 'package:flutter/material.dart';
import 'package:yueyou/core/theme/cyber_colors.dart';
import 'package:yueyou/core/theme/cyber_dimensions.dart';
import 'package:yueyou/core/theme/cyber_text_styles.dart';
import 'package:yueyou/shared/widgets/cyber_modal.dart';
/// 赛博朋克风格确认对话框
/// 用于重置游戏等需要二次确认的操作
/// 基于 showCyberModal 封装，内部只提供确认内容（标题 + 消息 + 按钮）
Future<bool> showCyberConfirmDialog({
  required BuildContext context,
  required String title,
  required String message,
  String confirmText = '确认',
  String cancelText = '取消',
}) async {
  final result = await showCyberModal<bool>(
    context: context,
    child: Padding(
      padding: const EdgeInsets.all(CyberDimensions.spacingL),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text(
            title,
            style: CyberTextStyles.dialogTitle.copyWith(
```

```
              color: CyberColors.whiteHigh,
              fontWeight: FontWeight.w900,
            ),
          ),
          const SizedBox(height: CyberDimensions.spacingM),
          Flexible(
            child: SingleChildScrollView(
              child: Text(
                message,
                textAlign: TextAlign.center,
                style: CyberTextStyles.bodySmall.copyWith(
                  color: CyberColors.whiteMedium,
                  fontSize: 14,
                  height: 1.5,
                ),
              ),
            ),
          ),
          const SizedBox(height: CyberDimensions.spacingL),
          Row(
            children: [
              Expanded(
                child: _CyberButton(
                  text: cancelText,
                  isPrimary: false,
                  onTap: () => Navigator.of(context).pop(false),
                ),
              ),
              const SizedBox(width: CyberDimensions.spacingMS),
              Expanded(
                child: _CyberButton(
                  text: confirmText,
                  isPrimary: true,
                  onTap: () => Navigator.of(context).pop(true),
                ),
              ),
            ],
          ),
        ],
      ),
    ),
  );
  return result ?? false;
}
class _CyberButton extends StatelessWidget {
  final String text;
  final bool isPrimary;
  final VoidCallback onTap;
  const _CyberButton({
    required this.text,
```

```
    required this.isPrimary,
    required this.onTap,
  });
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding:
            const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
        decoration: BoxDecoration(
          gradient: isPrimary
              ? const LinearGradient(
                  colors: [CyberColors.hotPink, CyberColors.neonPurple],
                  begin: Alignment.topLeft,
                  end: Alignment.bottomRight,
                )
              : null,
          color: isPrimary ? null : CyberColors.whiteSubtle,
          borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
          border: Border.all(
            color:
                isPrimary ? CyberColors.transparent : CyberColors.whiteSubtle,
            width: CyberDimensions.borderNormal,
          ),
        ),
        child: Center(
          child: Text(
            text,
            style: CyberTextStyles.buttonLabel.copyWith(
              color: CyberColors.whiteHigh,
              fontWeight: FontWeight.w900,
            ),
          ),
        ),
      ),
    );
  }
}

// ===== 文件：lib/shared/widgets/cyber_modal.dart =====
import 'dart:ui';
import 'package:flutter/material.dart';
import 'package:yueyou/core/theme/cyber_colors.dart';
import 'package:yueyou/core/theme/cyber_dimensions.dart';
import 'package:yueyou/core/utils/cyber_performance_detector.dart';
/// 赛博朋克风格全局弹窗封装
/// 替代 Navigator.push 全屏跳转，点击外部可关闭
Future<T?> showCyberModal<T>({
  required BuildContext context,
```

```
  required Widget child,
  bool barrierDismissible = true,
}) {
  return showGeneralDialog<T>(
    context: context,
    barrierDismissible: barrierDismissible,
    barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,
    barrierColor: CyberColors.blackOverlay,
    transitionDuration: CyberDimensions.animNormal,
    pageBuilder: (context, animation, secondaryAnimation) {
      return SafeArea(
        child: Center(
          child: Material(
            type: MaterialType.transparency,
            child: ScaleTransition(
              scale: CurvedAnimation(
                parent: animation,
                curve: Curves.easeOutCubic,
              ),
              child: FadeTransition(
                opacity: animation,
                child: Container(
                  margin: const EdgeInsets.symmetric(
                    horizontal: CyberDimensions.spacingXL,
                    vertical: 64,
                  ),
                  constraints: BoxConstraints(
                    maxWidth: 500,
                    maxHeight: MediaQuery.of(context).size.height * 0.85,
                  ),
                  decoration: BoxDecoration(
                    borderRadius:
                        BorderRadius.circular(CyberDimensions.radiusL),
                    border: Border.all(
                      color: CyberColors.neonCyan.withValues(alpha: 0.4),
                      width: CyberDimensions.borderThick,
                    ),
                    boxShadow: [
                      BoxShadow(
                        color: CyberColors.neonCyan.withValues(alpha: 0.25),
                        blurRadius: 30,
                        spreadRadius: 2,
                      ),
                      const BoxShadow(
                        color: CyberColors.blackShadow,
                        blurRadius: 40,
                        offset: Offset(0, 15),
                      ),
                    ],
                  ),
```

```
                    child: ClipRRect(
                      borderRadius: BorderRadius.circular(
                        CyberDimensions.radiusL - CyberDimensions.borderThick,
                      ),
                      child: CyberPerformanceDetector.detectLevel() ==
                              CyberAnimationLevel.low
                          ? Container(
                              color:
                                  CyberColors.glassDark.withValues(alpha: 0.98),
                              child: child, //  直接渲染内容
                            )
                          : BackdropFilter(
                              filter: ImageFilter.blur(
                                sigmaX: CyberDimensions.blurStrong,
                                sigmaY: CyberDimensions.blurStrong,
                              ),
                              child: Container(
                                color: CyberColors.glassDark,
                                child: child, //  移除了冗余的内部 ClipRRect，统一在外层精准截断
                              ),
                            ),
                    ),
                  ),
                ),
              ),
            ),
          ),
        );
      },
    );
}

// ===== 文件：lib/shared/widgets/cyber_toast.dart =====
import 'dart:async';
import 'package:flutter/material.dart';
import 'dart:ui';
import '../../../core/theme/cyber_colors.dart';
import '../../../core/theme/cyber_dimensions.dart';
import '../../../core/theme/cyber_text_styles.dart';
import '../../../core/utils/cyber_performance_detector.dart';
import '../../../features/game_2048/presentation/widgets/board_mascot.dart';
import '../../../main.dart';
enum ToastType {
  info,
  error,
  success,
}
class CyberToast {
  static OverlayEntry? _currentEntry;
  static Timer? _currentTimer;
```

```
0021    static void show(
0022      String message, {
0023      ToastType type = ToastType.info,
0024      Duration duration = CyberDimensions.toastDuration,
0025    }) {
0026      // 移除之前的 Toast
0027      _removeCurrentEntry();
0029      // 创建新的 OverlayEntry
0030      final overlay = globalNavigatorKey.currentState?.overlay;
0031      if (overlay == null) return;
0032      final entry = OverlayEntry(
0033        builder: (context) => _CyberToastWidget(
0034          message: message,
0035          type: type,
0036          onRemove: () => _currentEntry = null,
0037        ),
0038      );
0040      _currentEntry = entry;
0041      overlay.insert(entry);
0043      // 设置定时器，自动移除
0044      _currentTimer?.cancel();
0045      _currentTimer = Timer(duration, () {
0046        _removeCurrentEntry();
0047      });
0048    }
0050    static void _removeCurrentEntry() {
0051      if (_currentEntry != null) {
0052        _currentEntry?.remove();
0053        _currentEntry = null;
0054      }
0055      _currentTimer?.cancel();
0056      _currentTimer = null;
0057    }
0058  }
0060  class _CyberToastWidget extends StatefulWidget {
0061    final String message;
0062    final ToastType type;
0063    final VoidCallback onRemove;
0065    const _CyberToastWidget({
0066      required this.message,
0067      required this.type,
0068      required this.onRemove,
0069    });
0071    @override
0072    __CyberToastWidgetState createState() => __CyberToastWidgetState();
0073  }
0075  class __CyberToastWidgetState extends State<_CyberToastWidget>
0076      with SingleTickerProviderStateMixin {
0077    late AnimationController _controller;
0078    late Animation<Offset> _slideAnimation;
```

```
  late Animation<double> _fadeAnimation;
  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: CyberDimensions.animNormal,
      vsync: this,
    );
    _slideAnimation = Tween<Offset>(
      begin: const Offset(0, -1),
      end: Offset.zero,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.easeOut,
      ),
    );
    _fadeAnimation = Tween<double>(
      begin: 0.0,
      end: 1.0,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.easeOut,
      ),
    );
    _controller.forward();
  }
  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
  Color _getBorderColor() => switch (widget.type) {
        ToastType.error => CyberColors.neonPink,
        ToastType.success => CyberColors.neonGreen,
        ToastType.info => CyberColors.neonCyan
      };
  @override
  Widget build(BuildContext context) {
    return Positioned(
      top: MediaQuery.of(context).padding.top + CyberDimensions.spacingM,
      left: 0,
      right: 0,
      child: FadeTransition(
        opacity: _fadeAnimation,
        child: SlideTransition(
          position: _slideAnimation,
          child: Align(
            alignment: Alignment.topCenter,
```

```
            child: IntrinsicWidth(
              child: Container(
                constraints: BoxConstraints(
                  maxWidth: MediaQuery.of(context).size.width * 0.85,
                ),
                margin: const EdgeInsets.symmetric(
                  horizontal: CyberDimensions.spacingML,
                ),
                decoration: BoxDecoration(
                  borderRadius: BorderRadius.circular(CyberDimensions.radiusL),
                  border: Border.all(
                    color: _getBorderColor(),
                    width: CyberDimensions.borderThick,
                  ),
                  boxShadow: [
                    BoxShadow(
                      color: _getBorderColor().withValues(alpha: 0.3),
                      blurRadius: 16,
                      offset: const Offset(0, 4),
                    ),
                  ],
                ),
                child: ClipRRect(
                  borderRadius: BorderRadius.circular(
                    CyberDimensions.radiusL - CyberDimensions.borderThick,
                  ),
                  child: CyberPerformanceDetector.detectLevel() ==
                          CyberAnimationLevel.low
                      ? Container(
                          color: CyberColors.glassDark.withValues(alpha: 0.98),
                          child: _buildContent(),
                        )
                      : BackdropFilter(
                          filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),
                          child: Container(
                            color: CyberColors.glassDark,
                            child: _buildContent(),
                          ),
                        ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }

  Widget _buildContent() {
    return Padding(
      padding: const EdgeInsets.symmetric(
```

```
        horizontal: 16,
        vertical: 12,
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        crossAxisAlignment: CrossAxisAlignment.center,
        children: [
          // XIAOYO Avatar PFP
          Container(
            width: 44,
            height: 44,
            decoration: BoxDecoration(
              shape: BoxShape.circle,
              color: CyberColors.background.withValues(alpha: 0.8),
              border: Border.all(
                color: _getBorderColor().withValues(alpha: 0.7),
                width: 1,
              ),
              boxShadow: [
                BoxShadow(
                  color: _getBorderColor().withValues(alpha: 0.5),
                  blurRadius: 8,
                ),
              ],
            ),
            child: ClipOval(
              child: Transform.scale(
                scale: 0.65,
                // Transform.translate 去微调位置以确保脸部在圆圈正中心
                child: Transform.translate(
                  offset: const Offset(0, -5),
                  child: const IgnorePointer(
                    child: BoardMascot(),
                  ),
                ),
              ),
            ),
          ),
          const SizedBox(width: 14),
          // Text Section
          Flexible(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'XIAOYO',
                  style: CyberTextStyles.captionBold.copyWith(
                    color: _getBorderColor(),
                    fontSize: 10,
```

```
                    letterSpacing: 1.2,
                  ),
                ),
                const SizedBox(height: 4),
                Text(
                  widget.message,
                  style: CyberTextStyles.bodySmallBold.copyWith(
                    color: CyberColors.whiteHigh,
                    height: 1.3,
                  ),
                  softWrap: true,
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

// ===== 文件：lib/shared/widgets/safe_padding_wrap.dart =====
import 'package:flutter/material.dart';
import 'package:yueyou/core/theme/cyber_colors.dart';
import 'package:yueyou/core/utils/safe_area_utils.dart';
/// 顶部安全区包裹组件
/// 严格执行“呼吸感”准则：SafeArea + 额外 24.0px 的 Padding
class SafePaddingWrap extends StatelessWidget {
  final Widget child;
  const SafePaddingWrap({
    super.key,
    required this.child,
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      color: CyberColors.background,
      child: SafeArea(
        bottom: false, // 顶部组件不强制底部安全区
        child: Padding(
          padding: const EdgeInsets.only(
            top: SafeAreaUtils.topBreathPadding,
            left: 16.0,
            right: 16.0,
          ),
          child: child,
        ),
      ),
    );
  }
```

```
0032  }

// ===== 文件：lib/shared/widgets/tts_error_listener.dart =====
0001  import 'package:flutter/material.dart';
0002  import 'package:yueyou/core/utils/cyber_logger.dart';
0003  import 'package:flutter_riverpod/flutter_riverpod.dart';
0004  import '../../features/audio/domain/tts_audio_state.dart';
0005  import '../../features/audio/providers/tts_audio_notifier.dart';
0006  import 'cyber_toast.dart';
0008  /// TTS 错误全局监听器组件
0009  ///
0010  /// 将任意页面或整个 App 根节点包裹在此组件下，
0011  /// 每当 [TtsAudioError] 产生新时间戳时，
0012  /// 自动弹出 [CyberToast] 错误提示，无需各页面手动监听。
0013  class TtsErrorListener extends ConsumerStatefulWidget {
0014    final Widget child;
0015    const TtsErrorListener({super.key, required this.child});
0017    @override
0018    ConsumerState<TtsErrorListener> createState() => _TtsErrorListenerState();
0019  }
0021  class _TtsErrorListenerState extends ConsumerState<TtsErrorListener> {
0022    int _previousErrorTime = 0;
0023    String? _previousFallback;
0024    int _lastFallbackTimestamp = 0;
0026    @override
0027    void didChangeDependencies() {
0028      super.didChangeDependencies();
0029      // 依赖变化时刷新快照，避免初始化重复触发旧错误
0030      final audioState = ref.read(ttsAudioProvider);
0031      _previousErrorTime = switch (audioState) {
0032        TtsAudioError(:final timestamp) => timestamp,
0033        TtsAudioIdle() ||
0034        TtsAudioBuffering() ||
0035        TtsAudioPlaying() ||
0036        TtsAudioPaused() =>
0037          0,
0038      };
0039      _previousFallback = audioState.fallbackMessage;
0040    }
0042    @override
0043    Widget build(BuildContext context) {
0044      final audioState = ref.watch(ttsAudioProvider);
0045      final (:err, :timestamp) = switch (audioState) {
0046        TtsAudioError(:final message, :final timestamp) => (
0047            err: message,
0048            timestamp: timestamp,
0049          ),
0050        TtsAudioIdle() ||
0051        TtsAudioBuffering() ||
0052        TtsAudioPlaying() ||
```

```
      TtsAudioPaused() =>
        (err: null, timestamp: 0),
    };
    final currentTimestamp = DateTime.now().millisecondsSinceEpoch;
    if (err != null && timestamp != _previousErrorTime) {
      _previousErrorTime = timestamp;
      CyberLogger.captureMessage('[TtsErrorListener] 检测到新错误，将展示 CyberToast');
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!mounted) return;
        try {
          CyberToast.show(err, type: ToastType.error);
        } catch (e, stack) {
          CyberLogger.captureWarning(
            e,
            stack: stack,
            tag: 'tts',
            extra: {'context': 'TtsErrorListener 展示错误 Toast 失败'},
          );
        }
      });
    }
    final fallback = audioState.fallbackMessage;
    if (fallback == null) {
      _previousFallback = null;
    } else {
      final isSameFallback = fallback == _previousFallback;
      final isThrottled =
          isSameFallback && (currentTimestamp - _lastFallbackTimestamp < 1000);
      if (!isThrottled) {
        _previousFallback = fallback;
        _lastFallbackTimestamp = currentTimestamp;
        WidgetsBinding.instance.addPostFrameCallback((_) {
          if (!mounted) return;
          try {
            CyberToast.show(fallback, type: ToastType.info);
          } catch (e, stack) {
            CyberLogger.captureWarning(
              e,
              stack: stack,
              tag: 'tts',
              extra: {'context': 'TtsErrorListener 展示降级通知 Toast 失败'},
            );
          }
        });
      }
    }
    return widget.child;
  }
}
```